

DeerFlow 橙皮书

2.0 · 字节开源的 Super Agent Harness

花叔（的模仿）

2026-04-09

DeerFlow 橙皮书

作者：花叔（的模仿）

版本：v2.0

日期：2026-04-09

简介：DeerFlow 2.0 从入门到精通，深度解析字节开源的 Super Agent Harness

版权所有 © 2026 花叔（的模仿）。保留所有权利。
未经书面许可，不得以任何形式复制或传播本书内容。

目录

DeerFlow 橙皮书

Part 1: 认识 DeerFlow

01 DeerFlow 是什么

02 核心概念一览

03 从 Deep Research 到 Super Agent Harness

04 安装和环境要求

Part 2: 技术架构

05 架构概览：四组件协同

06 Lead Agent：主代理

07 Sandbox：隔离执行环境

08 Sub-Agents：子代理系统

09 Skills：能力模块

10 Memory System：长期记忆

11 配置详解：config.yaml 与 extensions_config.json

Part 3: 部署方案

12 本地开发部署

13 Docker 部署（推荐）

14 生产环境部署

15 性能调优

Part 4: 核心功能

16 Web 界面使用

17 IM 渠道集成

18 Claude Code 集成

19 文件上传和文档处理

Part 5: Skills 进阶

20 内置 Skills 概览

21 自定义 Skills 开发

注意事项

22 MCP Server 集成

Part 6: 开发指南

23 内嵌 Python Client

24 自定义 Agent 开发

附录

阅读指南

DeerFlow 橙皮书

2.0 · 字节开源的 Super Agent Harness

Part 1: 认识 DeerFlow

这Part讲清楚 DeerFlow 是什么、为什么重要、和之前的方案有什么不同。

01 DeerFlow 是什么

01.1 先给结论

DeerFlow 是字节跳动开源的 **Super Agent Harness**。

不是框架，不是 SDK。是让 Agent 真正把事情做完的运行时基础设施。

2026年2月28日发布，发布当天登上 GitHub Trending 第1名。

01.2 这节讲什么

这节讲三件事：

1. DeerFlow 的定位（harness vs framework）
2. 它解决什么问题
3. 为什么值得关注

01.3 Harness 是什么意思

我第一次看到 “harness” 这个词，愣了一下。

Harness 原意是“马具”——套在马身上的装备，让马能拉车、能干活。

DeerFlow 的定位就是这样：它不是教你怎么造 Agent 的框架，而是套在 Agent 身上的装备，让 Agent 能干活、能完成任务。

框架是教你造车。Harness 是给你一辆能跑的车。

01.4 它解决什么问题

我用过很多 Agent 工具。大部分有个共同问题：**只会说，不会做**。

Agent 说“我来帮你生成一份报告”。然后它真的能生成吗？不一定。

- 没有文件系统，没法写文件
- 没有 sandbox，没法执行代码
- 没有 memory，记不住你的偏好
- 没有 sub-agents，复杂任务一次跑不完

DeerFlow 解决的就是这些问题。它给 Agent 配了一套完整的"干活装备":

能力	传统 Agent	DeerFlow
文件系统	无	有 (sandbox内)
执行代码	无或受限	有 (Docker隔离)
记忆	无	有 (跨session持久)
子任务	无	有 (最多3个并发)
扩展能力	有限	有 (Skills + MCP)

花叔的经验: Agent 的价值不在于"能说", 在于"能做"。

我测过十几种 Agent 工具。能真正完成复杂任务的不到 20%。大多数停在"我会帮你"这句话。DeerFlow 是少数真的能把事情做完的。

01.5 为什么值得关注

三个原因:

第一，字节开源。字节在国内 AI 应用落地方面有实战经验。开源的代码通常经过真实业务验证。

第二，架构设计。LangGraph + LangChain 的组合是目前 Agent 编排的主流方案。DeerFlow 选对了技术栈。

第三，生态潜力。22个内置 Skills、MCP Server 集成、IM 渠道支持。这不是玩具，是生产级系统。

01.6 适合谁用

- **开发者:** 想用 Agent 完成复杂任务，不只是聊天
- **技术管理者:** 评估企业级 Agent 平台选型
- **AI应用架构师:** 研究 Agent 系统设计模式

不适合:

- 纯聊天场景 (ChatGPT 更简单)
- 不想折腾配置的人 (需要 Docker、API Key)

- 只想快速体验的人（本地部署至少需要 4核8GB）

01.7 向前桥接

知道了 DeerFlow 是什么。下一节，看它的核心概念。

02 核心概念一览

02.1 先给结论

DeerFlow 有5个核心概念。记住这5个，就理解了整个系统：

1. **Lead Agent**：主代理，负责任务规划和执行
2. **Sub-Agents**：子代理，并行处理拆分后的子任务
3. **Sandbox**：隔离执行环境，有完整文件系统
4. **Skills**：能力模块，定义怎么做某类任务
5. **Memory**：长期记忆，记住你的偏好和背景

02.2 这节讲什么

这节用类比把5个概念串起来。不求完整，求理解。

02.3 一个类比：公司架构

把 DeerFlow 想象成一家公司：

DeerFlow 概念	公司类比
Lead Agent	项目经理
Sub-Agents	执行团队
Sandbox	办公室和工位
Skills	工作手册
Memory	员工档案

Lead Agent（项目经理）：接到任务后，规划怎么做、拆分成子任务、调度资源、汇总结果。它不亲自做所有事，但负责整体把控。

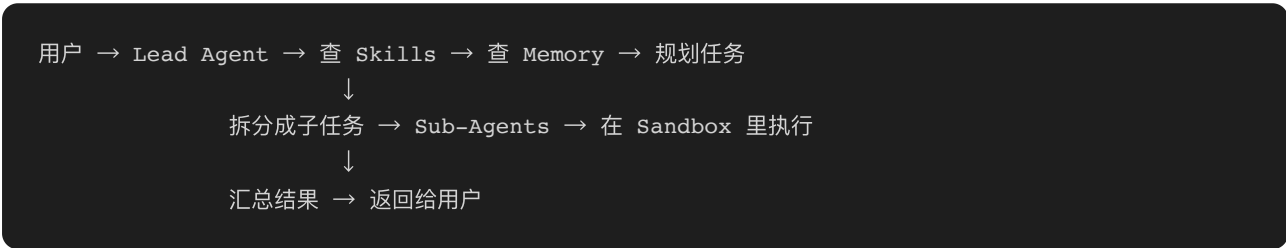
Sub-Agents（执行团队）：项目经理把大任务拆成小任务，分给不同的人做。每个人独立完成，最后项目经理汇总。最多同时3个人干活。

Sandbox（办公室）：干活要有地方。Sandbox 就是干活的地方。里面有文件柜、电脑、打印机。每个人有独立的工作空间，不会互相干扰。

Skills（工作手册）：怎么做PPT？怎么写报告？怎么做数据分析？这些"怎么做"的经验写在手册里。项目经理拿到任务后，会翻手册找方法。

Memory（员工档案）：你之前做过什么、喜欢什么风格、有什么习惯。这些信息记在档案里。下次你再来，项目经理翻档案就知道怎么配合你。

02.4 五个概念的关系



这个流程是 DeerFlow 的核心工作方式。

02.5 和传统 Agent 的区别

维度	传统 Agent	DeerFlow
执行者	单一 Agent	Lead + Sub-Agents
执行环境	无或受限	完整 Sandbox
方法来源	算法生成	Skills 定义
记忆	无	跨 Session 持久
并行能力	无	最多3并发

传统 Agent 是"单兵作战"。DeerFlow 是"团队协作"。

花叔的经验：理解 Agent 系统，关键是理解"谁来干活"和"在哪干活"。

大多数 Agent 只解决了"谁来干活"（一个 LLM）。DeerFlow 解决了"在哪干活"（Sandbox）和"怎么干活"（Skills）。这才是真正能把事情做完的原因。

02.6 概念依赖关系

要理解 Lead Agent，需要先理解 Sandbox（因为它在里面干活）。要理解 Sub-Agents，需要先理解 Lead Agent（因为它是 Lead Agent 拉起来的）。要理解 Skills，需要先理解 Lead Agent（因为它是 Lead Agent 查阅的）。要理解 Memory，它是独立的（但会被 Lead Agent 注入到 system prompt）。

建议阅读顺序：Sandbox → Lead Agent → Sub-Agents → Skills → Memory。

02.7 向前桥接

5个概念记住了。下一节，看 DeerFlow 怎么从 Deep Research 演化而来。

03 从 Deep Research 到 Super Agent Harness

03.1 先给结论

DeerFlow 1.x 是 Deep Research 框架。DeerFlow 2.0 是 Super Agent Harness。

两者不共用代码。2.0 是从头重写。

03.2 这节讲什么

这节讲 DeerFlow 的演化过程。理解"为什么重写"，就能理解 2.0 的设计逻辑。

03.3 起点：Deep Research

DeerFlow 最初定位是"深度研究框架"。

核心场景：帮用户做深度研究，收集信息、分析数据、生成报告。

这是单场景工具。设计时没考虑其他用途。

03.4 转折：用户用它做了什么

上线之后，开发者拿它做的事情远不止研究：

实际用途	占比（估算）
深度研究	30%
数据流水线	20%
演示文稿制作	15%
快速起 Dashboard	10%
自动化内容流程	10%
其他	15%

一开始连作者都没想到这些场景。

03.5 意识到的问题

用户反馈揭示了一个问题：DeerFlow 1.x 的架构限制它做更多事。

问题1：单一场景设计。研究流程是硬编码的，不适合其他场景。

问题2：拼装式架构。用户需要自己拼装组件，上手门槛高。

问题3：扩展能力有限。想加新功能，得改核心代码。

作者意识到：DeerFlow 不只是研究工具。它更像一个能让 Agent 完成任何任务的运行时基础设施。

03.6 重写的决定

2025年底，作者决定从头重写。

重写的核心目标：

目标	1.x	2.0
定位	研究框架	Super Agent Harness
架构	拼装式	开箱即用
扩展	改代码	Skills + MCP
场景	研究专用	通用任务

重写时间：大约3个月（2025年底到2026年2月）。

03.7 2.0 的变化

架构层面：

- 基于 LangGraph + LangChain（主流 Agent 编排方案）
- Harness/App 分层（核心框架独立，应用层可扩展）
- 内嵌 Python Client（不启动 HTTP 服务也能用）

功能层面：

- 22个内置 Skills（研究、报告、PPT、网页、图像等）
- Sub-Agents 系统（复杂任务拆分执行）
- Sandbox 执行环境（Docker 隔离）
- 长期 Memory（跨 Session 持久化）

渠道层面：

- IM 渠道集成（Telegram、Slack、飞书、企业微信）
- Claude Code 集成（终端里直接用）

03.8 为什么要关注 2.0

如果你用过 1.x，2.0 不是升级版。是全新产品。

如果你想用 DeerFlow，直接用 2.0。不要从 1.x 开始。

如果你在评估 Agent 平台，2.0 的架构设计值得研究。它是少数真正把"Agent 能完成任务"做出来的系统。

花叔的经验：产品演化比产品发布更有价值。

从 Deep Research 到 Super Agent Harness 的演化，揭示了 Agent 落地的真实需求：用户要的不是"研究工具"，是"能完成任务的 Agent"。这个认知转变，才是 DeerFlow 2.0 的核心价值。

03.9 向前桥接

理解了 DeerFlow 的演化。下一节，看怎么安装它。

04 安装和环境要求

04.1 先给结论

DeerFlow 有两种安装方式：

1. **Docker**（推荐）：一条命令跑起来
2. **本地开发**：需要 Python 3.12+、Node.js 22+、uv、pnpm

最低配置：4核 CPU、8GB 内存、20GB SSD。

04.2 这节讲什么

这节讲：

1. 系统要求
2. Docker 安装方式
3. 本地开发方式
4. Coding Agent 安装方式（给 Claude Code/Cursor 等用）

04.3 系统要求

部署场景	最低配置	推荐配置
本地体验	4核 / 8GB / 20GB SSD	8核 / 16GB
Docker 开发	4核 / 8GB / 25GB SSD	8核 / 16GB
长期服务	8核 / 16GB / 40GB SSD	16核 / 32GB

注意：

- macOS 和 Windows 更适合作为开发机或体验环境
- 长期运行更推荐 Linux + Docker
- 上面的配置只覆盖 DeerFlow 本身，本地大模型需额外预留资源

04.4 Docker 安装（推荐）

前提：已安装 Docker。

步骤：

第一步，克隆仓库：

```
git clone https://github.com/bytedance/deer-flow.git
cd deer-flow
```

第二步，生成配置文件：

```
make config
```

预期结果：项目根目录生成 `config.yaml` 和 `.env`。

第三步，配置模型：

编辑 `config.yaml`，定义至少一个模型：

```
models:
  - name: gpt-4
    display_name: GPT-4
    use: langchain_openai:ChatOpenAI
    model: gpt-4
    api_key: $OPENAI_API_KEY
    max_tokens: 4096
    temperature: 0.7
```

第四步，设置 API Key：

编辑 `.env`：

```
OPENAI_API_KEY=your-openai-api-key
TAVILY_API_KEY=your-tavily-api-key
```

第五步，启动服务：

```
make docker-init    # 拉取 sandbox 镜像 (首次执行)
make docker-start   # 启动服务
```

预期结果：访问 <http://localhost:2026> 看到 DeerFlow 界面。

注意：config.yaml 配置复杂

首次配置建议先用默认模型（GPT-4）。后续可以换成 Doubao、DeepSeek、Kimi。配置方法见第11节。

04.5 本地开发安装

前提：

- Python 3.12+
- Node.js 22+
- uv（Python 包管理）
- pnpm（Node 包管理）
- nginx（反向代理）

步骤：

第一步，克隆仓库：

```
git clone https://github.com/bytedance/deer-flow.git
cd deer-flow
```

第二步，检查依赖环境：

```
make check
```

预期结果：输出各项依赖的版本信息。

第三步，生成配置文件：

```
make config
```

第四步，配置模型和 API Key（同 Docker 安装）。

第五步，安装依赖：

```
make install
```

预期结果：安装 backend 和 frontend 所有依赖。

第六步，启动服务：

```
make dev
```

预期结果：4个进程启动（LangGraph、Gateway、Frontend、Nginx）。

访问地址：<http://localhost:2026>

04.6 Coding Agent 安装方式

如果你在用 Claude Code、Codex、Cursor、Windsurf，可以直接把这句话发给它：

```
如果还没 clone DeerFlow, 就先 clone, 然后按照 https://raw.githubusercontent.com/bytedance/deer-flow
```

Coding Agent 会自动：

1. Clone 仓库（如果还没）
2. 优先选择 Docker
3. 完成初始化
4. 告诉你下一条启动命令和还缺哪些配置

这是最省心的方式。

04.7 Windows 用户注意

本地开发模式在 Windows 上需要用 Git Bash。

基于 bash 的服务脚本不支持直接在原生 cmd.exe 或 PowerShell 中执行。

WSL 也不保证可用（部分脚本依赖 Git for Windows 的 cygpath 等工具）。

推荐：Windows 用户用 Docker 方式。

04.8 配置文件位置

配置文件	推荐位置	用途
config.yaml	项目根目录	主配置（模型、工具、sandbox）
extensions_config.json	项目根目录	MCP 和 Skills 配置
.env	项目根目录	环境变量（API Key）

04.9 常见问题

Q：make config 报错？

A：检查 Makefile 是否在当前目录。确保在项目根目录执行。

Q：make docker-start 启动失败？

A：检查 Docker 是否运行。检查 config.yaml 中的模型配置是否正确。

Q：访问 localhost:2026 无响应？

A：检查 nginx 是否启动。检查 4 个进程是否都在运行（LangGraph 2024、Gateway 8001、Frontend 3000、Nginx 2026）。

花叔的经验：安装是第一道坎，跨过去就顺利了。

我测过很多开源项目。DeerFlow 的安装流程算比较顺的。主要坑在配置文件（config.yaml 比较长）。建议首次安装先用默认配置跑通，再逐步调整。

04.10 向前桥接

安装完成。下一Part，拆解 DeerFlow 的技术架构。

Part 2: 技术架构

这Part拆解 DeerFlow 的技术骨架。每节一个核心组件。

05 架构概览：四组件协同

05.1 先给结论

DeerFlow 由4个组件构成：

1. **LangGraph Server**（端口2024）：Agent 运行时
2. **Gateway API**（端口8001）：REST API 入口
3. **Frontend**（端口3000）：Web 界面
4. **Nginx**（端口2026）：统一入口

所有请求走 Nginx，Nginx 分流到其他组件。

05.2 这节讲什么

这节讲：

1. 四组件的分工
2. 请求路由流程
3. 为什么这样设计

05.3 四组件分工

组件	端口	负责什么
LangGraph Server	2024	Agent 执行、状态管理、工具调用
Gateway API	8001	模型列表、MCP配置、Skills管理、Memory、文件上传
Frontend	3000	用户界面、聊天交互、文件上传、结果展示
Nginx	2026	反向代理、统一入口、路由分流

LangGraph Server：这是核心。Agent 在这里运行。它处理：

- 用户消息的理解和响应
- 工具调用（bash、read_file、web_search等）

- Sub-Agent 的调度
- 状态持久化 (thread、checkpoint)

Gateway API: 这是管理端。它处理:

- 查询和配置模型列表
- 管理 MCP Server
- 管理 Skills (启用/禁用/安装)
- Memory 数据的读写
- 文件上传 (PDF/PPT/Excel转Markdown)

Frontend: 这是用户端。它提供:

- 聊天界面
- 模型选择
- 文件上传
- Thread 管理
- 结果展示 (报告、PPT、网页等)

Nginx: 这是入口。它负责:

- 把 `/api/langgraph/*` 路到 LangGraph
- 把 `/api/*` (其他) 路到 Gateway
- 把 `/` (非API) 路到 Frontend
- 统一端口2026, 简化访问

05.4 请求路由流程

```
用户 → Nginx(2026)
      ↓
      路径判断:
      |—— /api/langgraph/* → LangGraph(2024)
      |—— /api/* (其他)    → Gateway(8001)
      |—— /                 → Frontend(3000)
```

聊天请求:


```
用户发消息 → POST /api/langgraph/threads/{id}/runs
            → Nginx 转发到 LangGraph
            → LangGraph 执行 Agent
            → 返回结果给用户
```

文件上传:

```
用户上传文件 → POST /api/threads/{id}/uploads
              → Nginx 转发到 Gateway
              → Gateway 调用 markdown 转换
              → 存到 sandbox 目录
              → 返回文件列表给用户
```

查询模型列表:

```
用户点击模型 → GET /api/models
              → Nginx 转发到 Gateway
              → Gateway 读取 config.yaml
              → 返回模型列表给用户
```

05.5 为什么这样设计

问题1: 为什么要有 Gateway?

Agent 运行时 (LangGraph) 只负责执行。管理功能 (模型、MCP、Skills、Memory) 需要单独的 API。

如果把管理功能塞进 LangGraph, 会耦合太重。Gateway 做管理, LangGraph 做执行, 职责清晰。

问题2: 为什么用 Nginx?

简化访问。用户只需要知道一个地址: localhost:2026。

如果不用 Nginx, 用户需要记住:

- 聊天走 2024
- 上传走 8001
- 界面走 3000

太复杂。Nginx 把这些统一到一个端口。

问题3: 为什么 LangGraph 单独进程?

Agent 执行是 CPU 密集型。单独进程可以：

- 独立管理并发
- 独立重启（故障隔离）
- 独立扩展（多实例）

Gateway 是轻量 API，不需要这些。

05.6 Gateway Mode（实验性）

还有一种运行模式：Gateway Mode。

Agent 运行时嵌入 Gateway，不启动 LangGraph Server。

命令：`make dev-pro` 或 `make start-pro`

优点：进程数减少（4变3），资源占用更低。

缺点：Agent 执行和 Gateway 共享进程，故障隔离差。

目前是实验性功能。生产环境建议用标准模式。

05.7 Provisioner（可选）

如果 sandbox 用 Kubernetes 模式，还需要 Provisioner 服务。

端口：8002。

它负责：

- 在 K8s 里创建 Pod 作为 sandbox
- 管理 Pod 生命周期
- 执行命令、读写文件

Provisioner 只在 `config.yaml` 配置了 `provisioner_url` 时才启动。

05.8 端口速查

端口	组件	启动条件
2026	Nginx	总是启动
3000	Frontend	总是启动
8001	Gateway	总是启动
2024	LangGraph	标准模式启动
8002	Provisioner	K8s sandbox 配置时启动

花叔的经验：理解架构的关键是理解"请求怎么流转"。

把请求从用户到最终处理的路径画出来，就知道每个组件负责什么了。DeerFlow 的设计是"执行和管理分离"，这是大型系统的常见模式。

05.9 向前桥接

架构概览清楚了。下一节，深入 Lead Agent。

06 Lead Agent：主代理

06.1 先给结论

Lead Agent 是 DeerFlow 的核心。它负责：

1. 接收用户任务
2. 规划执行步骤
3. 调用工具执行
4. 调度 Sub-Agents
5. 汇总最终结果

它不是单点，是一整套系统：Agent Factory + System Prompt + Middlewares + Tools。

06.2 这节讲什么

这节拆解 Lead Agent 的构成：

1. Agent Factory
2. System Prompt 生成
3. Middleware Chain（12个中间件）
4. Tools 加载机制

06.3 Agent Factory

Lead Agent 通过 `make_lead_agent()` 函数创建。

位置： `packages/harness/deerflow/agents/lead_agent/agent.py`

创建过程：

```
def make_lead_agent(config: RunnableConfig):
    # 1. 创建模型 (根据 config 选择)
    model = create_chat_model(config)

    # 2. 加载工具 (sandbox、MCP、skills、subagent)
    tools = get_available_tools(...)

    # 3. 生成 system prompt (注入 skills、memory)
    prompt = apply_prompt_template(state)

    # 4. 构建中间件链
    middlewares = build_middlewares(config)

    # 5. 创建 agent
    agent = create_react_agent(model, tools, prompt, middlewares)

    return agent
```

关键点：

- 模型可动态选择（通过 `model_name` 配置）
- 工具按需加载（skills、MCP、subagent 可启用/禁用）
- System Prompt 动态生成（注入当前状态）
- 中间件链接按配置组装

06.4 System Prompt 结构

System Prompt 由多个部分组成：

```

<identity>
你是一个超级Agent...
</identity>

<skills>
[当前启用的 skills 列表, 带路径]
</skills>

<memory>
[用户的长期记忆 facts]
</memory>

<subagents>
[如果启用, subagent 使用说明]
</subagents>

<instructions>
[执行规则、输出格式]
</instructions>

```

Skills 部分:

```

<skills>
以下 skills 可用:
- /mnt/skills/public/deep-research/SKILL.md
- /mnt/skills/public/ppt-generation/SKILL.md
- /mnt/skills/public/frontend-design/SKILL.md

加载方式: 用 read_file 工具读取对应 SKILL.md 文件。
</skills>

```

Skills 按需渐进加载。不会一次性全塞进上下文。

Memory 部分:

```

<memory>
用户偏好:
- 喜欢用图表呈现数据
- 写作风格偏好短句
- 常用技术栈: Python、LangChain

近期任务:
- 做过3次深度研究报告
- 生成了2份演示文稿
</memory>

```

Memory 从 `memory.json` 文件读取，注入到 prompt。

06.5 Middleware Chain（12个中间件）

Lead Agent 有12个中间件，按顺序执行：

序号	Middleware	功能	条件
1	ThreadDataMiddleware	创建 thread 目录	总是
2	UploadsMiddleware	注入上传文件	总是
3	SandboxMiddleware	获取 sandbox	总是
4	DanglingToolCallMiddleware	修复缺失响应	总是
5	GuardrailMiddleware	工具调用授权	可选
6	SummarizationMiddleware	上下文压缩	可选
7	TodoListMiddleware	任务追踪	plan_mode
8	TitleMiddleware	生成标题	总是
9	MemoryMiddleware	记忆更新队列	总是
10	ViewImageMiddleware	注入图片数据	vision模型
11	SubagentLimitMiddleware	限制并发数	subagent启用
12	ClarificationMiddleware	拦截澄清请求	总是（最后）

执行顺序的意义：

前6个准备执行环境（thread、sandbox、文件、压缩）。中间3个处理输出（标题、记忆、图片）。最后2个限制和拦截（并发限制、澄清拦截）。

关键中间件详解：

ThreadDataMiddleware：

- 为每个 thread 创建独立目录

- 目 录 结 构 : `backend/.deer-flow/threads/{thread_id}/user-data/{workspace,uploads,outputs}`
- 确保不同 thread 的数据隔离

SandboxMiddleware:

- 从 SandboxProvider 获取 sandbox
- 存储 sandbox_id 到 state
- 后续工具调用都通过这个 sandbox

MemoryMiddleware:

- 过滤消息（只保留用户输入 + 最终AI响应）
- 推入更新队列
- 后台异步更新 memory.json

ViewImageMiddleware:

- 检查模型是否支持 vision
- 如果支持，注入 base64 图片数据到 state
- Agent 能"看见"图片

06.6 Tools 加载机制

工具按 groups 加载:

```
get_available_tools(
    groups=["sandbox", "builtin", "mcp", "community"],
    include_mcp=True,
    model_name="gpt-4",
    subagent_enabled=True
)
```

加载顺序:

1. **Config-defined tools**: 从 config.yaml 的 `tools[]` 配置
2. **MCP tools**: 从 extensions_config.json 的 MCP Server
3. **Built-in tools**:
 - `present_files`: 展示输出文件
 - `ask_clarification`: 请求澄清（会被拦截）

- `view_image`：读取图片（vision模型）

4. Subagent tool: `task`（如果启用）

工具分组：

Group	包含的工具
sandbox	bash, ls, read_file, write_file, str_replace
builtin	present_files, ask_clarification, view_image
mcp	MCP Server 提供的工具
community	tavily, jina_ai, firecrawl, image_search

06.7 运行时配置

Agent 行为通过 `config.configurable` 控制：

配置项	含义	效果
thinking_enabled	启用扩展思考	模型用 thinking 模式
model_name	模型名称	选择特定模型
is_plan_mode	规划模式	启用 TodoList
subagent_enabled	子代理模式	启用 task 工具

这些配置可通过：

- Web 界面的模式切换
- IM 渠道的 session 配置
- API 请求的 configurable 参数

06.8 ThreadState

每个 thread 有独立状态：

```
class ThreadState(AgentState):
    sandbox: str          # sandbox_id
    thread_data: dict      # thread 目录信息
    title: str            # thread 标题
    artifacts: list        # 输出文件列表
    todos: list           # 任务列表 (plan mode)
    uploaded_files: list   # 上传文件列表
    viewed_images: list    # 已查看图片
```

状态通过 LangGraph 的 checkpoint 持久化。下次对话能恢复。

花叔的经验：理解 Agent 的关键是理解"状态怎么流转"。

Lead Agent 不是单次请求响应。它是状态机。每个中间件处理状态的一部分，工具调用改变状态，最终状态持久化。这才是 Agent 能做复杂任务的原因。

06.9 向前桥接

Lead Agent 清楚了。下一节，看它在什么环境里执行：Sandbox。

07 Sandbox：隔离执行环境

07.1 先给结论

Sandbox 是 Agent 的"工作间"。

里面有完整的文件系统、能执行代码、能读写文件。而且是隔离的——每个 thread 有独立的 sandbox，不会互相干扰。

这是"会聊天的 Agent"和"能做事的 Agent"的核心区别。

07.2 这节讲什么

这节讲：

1. Sandbox 是什么
2. 三种执行模式
3. 虚拟路径系统
4. Sandbox 工具

07.3 Sandbox 是什么

Sandbox 是一个隔离的执行环境。

它解决三个问题：

问题1：Agent 怎么写文件？

传统 Agent 只能"说"要写什么文件。DeerFlow 的 Agent 在 sandbox 里真的能写。

问题2：Agent 怎么执行代码？

传统 Agent 不能执行代码。DeerFlow 的 Agent 在 sandbox 里能跑 bash、能执行 Python。

问题3：不同任务怎么隔离？

如果多个 thread 共用同一个文件系统，会互相污染。Sandbox 给每个 thread 分配独立空间。

07.4 三种执行模式

模式	执行位置	适用场景
Local	宿主机	开发调试、低风险任务
Docker	Docker容器	生产环境、高隔离要求
Kubernetes	K8s Pod	大规模部署、弹性伸缩

配置方式：

```
# config.yaml
sandbox:
  use: deerflow.community.aio_sandbox:AioSandboxProvider
```

Local 模式：

```
sandbox:
  use: deerflow.sandbox.local:LocalSandboxProvider
```

直接在宿主机文件系统执行。不隔离。

适合开发调试，不适合生产。

Docker 模式：

```
sandbox:
  use: deerflow.community.aio_sandbox:AioSandboxProvider
# 默认使用 Docker
```

在 Docker 容器里执行。每个 thread 一个容器。

适合生产环境。隔离性好。

Kubernetes 模式：

```
sandbox:
  use: deerflow.community.aio_sandbox:AioSandboxProvider
  provisioner_url: http://provisioner:8002
```

通过 Provisioner 服务在 K8s Pod 里执行。

适合大规模部署。Pod 可以动态创建和销毁。

07.5 虚拟路径系统

Agent 看到的路径不是宿主机的真实路径。

虚拟路径：

```
/mnt/user-data/
├── uploads/      # 用户上传的文件
├── workspace/    # Agent 的工作目录
└── outputs/      # 最终输出文件

/mnt/skills/
├── public/       # 内置 skills
└── custom/       # 自定义 skills

/mnt/acp-workspace/ # ACP agent 工作区（只读）
```

真实路径（Docker模式）：

```
backend/.deer-flow/threads/{thread_id}/user-data/
├── uploads/
├── workspace/
└── outputs/

deer-flow/skills/
├── public/
└── custom/
```

路径翻译：

Agent 调用工具时，虚拟路径自动翻译成真实路径。

例如：

- Agent 读 `/mnt/user-data/uploads/report.pdf`

- Sandbox 翻译成 `backend/.deer-flow/threads/{thread_id}/user-data/uploads/report.pdf`

这样做的好处：

- Agent 不需要知道真实路径
- 不同 sandbox 有不同的真实路径映射
- 迁移部署时 Agent 代码不变

07.6 Sandbox 工具

Sandbox 提供5个核心工具：

工具	功能	示例
bash	执行命令	<code>bash("ls -la /mnt/user-data")</code>
ls	列目录	<code>ls("/mnt/user-data/workspace")</code>
read_file	读文件	<code>read_file("/mnt/user-data/uploads/report.md")</code>
write_file	写文件	<code>write_file("/mnt/user-data/workspace/draft.md", content)</code>
str_replace	替换	<code>str_replace(path, old, new)</code>

bash 工具：

```
bash(command="pip install pandas")
bash(command="python analyze.py")
bash(command="ls -la /mnt/user-data")
```

执行结果返回：

- stdout: 命令输出
- stderr: 错误信息
- return_code: 退出码

read_file 工具：

```
read_file(  
  path="/mnt/user-data/uploads/report.md",  
  start_line=1,  
  end_line=100  
)
```

支持分段读取。大文件不用一次性读完。

write_file 工具:

```
write_file(  
  path="/mnt/user-data/workspace/output.md",  
  content="# 分析报告\n\n...",  
  mode="write" # 或 "append"  
)
```

支持追加模式。可以多次写入同一个文件。

str_replace 工具:

```
str_replace(  
  path="/mnt/user-data/workspace/draft.md",  
  old="## 旧标题",  
  new="## 新标题",  
  replace_all=False # True 则替换所有匹配  
)
```

用于文件内容的精准替换。

07.7 Sandbox 生命周期

```
Thread 创建
  ↓
SandboxMiddleware.acquire()
  ↓
获取 sandbox_id
  ↓
工具调用 (通过 sandbox)
  ↓
...
  ↓
Thread 结束或超时
  ↓
Sandbox.release(sandbox_id)
```

Docker 模式:

- acquire: 启动容器, 挂载 volume
- release: 停止容器, 清理资源

K8s 模式:

- acquire: 创建 Pod, 等待就绪
- release: 删除 Pod, 释放资源

Local 模式:

- acquire: 创建目录
- release: 目录保留 (下次可能复用)

07.8 文件流转

```
用户上传 → Gateway → markdown转换 → 存入 uploads/
  ↓
Agent read_file(uploads/) → 分析内容
  ↓
Agent write_file(workspace/) → 中间文件
  ↓
Agent write_file(outputs/) → 最终输出
  ↓
present_files(outputs/) → 展示给用户
```


用户上传的文件进入 `uploads/`。Agent 在 `workspace/` 里工作。最终结果写入 `outputs/`。用户在 Web 界面看到 `outputs/` 的文件。

07.9 Sandbox 与安全

Sandbox 是隔离环境，但不是安全边界。

注意事项：

- Agent 执行的命令在 sandbox 内，可能仍有风险
- 生产环境建议用 Docker 模式，不要用 Local
- Kubernetes 模式需要正确配置 Pod 安全策略

DeerFlow 官方建议：部署在本地可信环境（127.0.0.1）。

如果要公网部署，必须加安全措施：

- IP 白名单
- 前置身份验证
- 网络隔离

花叔的经验：Sandbox 是 Agent 落地的关键基础设施。

我用过很多 Agent，能真正处理文件的不到10%。大多数停在"我会帮你生成"这句话。DeerFlow 的 Sandbox 是真的有地方写文件、执行代码。这才是"Agent 能完成任务"的硬件基础。

07.10 向前桥接

Sandbox 清楚了。下一节，看 Sub-Agents 怎么并行执行。

08 Sub-Agents: 子代理系统

08.1 先给结论

Sub-Agents 是 DeerFlow 处理复杂任务的核心机制。

Lead Agent 把大任务拆成小任务，分给 Sub-Agents 并行执行。每个 Sub-Agent 独立上下文，独立工具，最后 Lead Agent 汇总结果。

最多3个 Sub-Agent 并发。15分钟超时。

08.2 这节讲什么

这节讲：

1. 为什么需要 Sub-Agents
2. Sub-Agent 的架构
3. 并发限制机制
4. 任务拆分和汇总

08.3 为什么需要 Sub-Agents

复杂任务单次执行不完。

例如：“帮我做一份完整的行业分析报告”

这个任务包含：

- 收集行业数据（可能要查10个来源）
- 分析竞品（可能要对比5家公司）
- 生成图表（可能要10张图）
- 撰写报告（可能要50页）
- 制作PPT（可能要20页）

如果 Lead Agent 顺序执行，可能需要几小时。

Sub-Agents 让这些步骤并行：

- Sub-Agent 1: 收集数据

- Sub-Agent 2: 分析竞品
- Sub-Agent 3: 生成图表

然后 Lead Agent 汇总，继续写报告、做PPT。

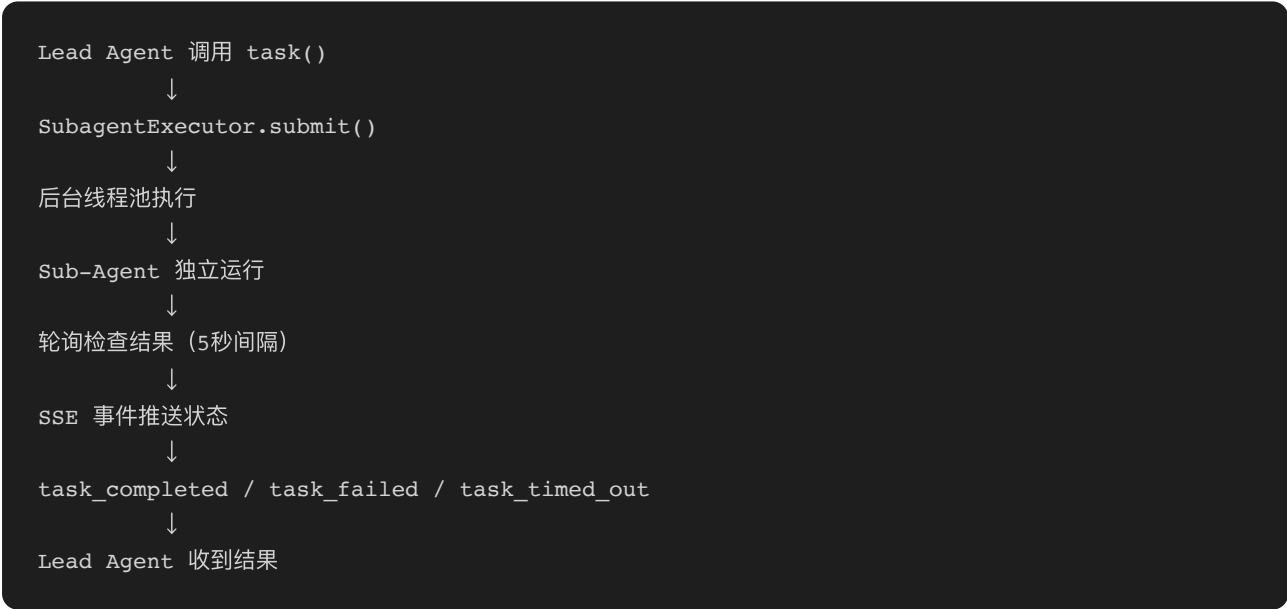
时间从几小时缩短到几十分钟。

08.4 Sub-Agent 架构

组件：

组件	位置	功能
task 工具	sandbox/tools.py	创建任务委托
Executor	subagents/executor.py	后台执行引擎
Registry	subagents/registry.py	Agent 注册表
Built-ins	subagents/builtins/	内置 Sub-Agent

流程：



08.5 task 工具

```
task(  
    description="收集新能源汽车行业数据",  
    prompt="搜索并整理2025年新能源汽车市场规模、主要厂商、技术趋势",  
    subagent_type="general-purpose",  
    max_turns=10  
)
```

参数:

- `description`: 任务描述 (显示给用户)
- `prompt`: 具体指令 (发给 Sub-Agent)
- `subagent_type`: Sub-Agent 类型
- `max_turns`: 最大对话轮数

返回:

- `task_started`: 任务开始
- `task_running`: 正在执行
- `task_completed`: 成功完成 (含结果)
- `task_failed`: 执行失败
- `task_timed_out`: 超时

08.6 Sub-Agent 类型

内置类型:

类型	工具集	适用场景
general-purpose	除 task 外所有工具	通用任务
bash	bash 工具专用	命令执行密集

general-purpose:

拥有 Lead Agent 的所有工具, 除了 `task` (不能嵌套调用 Sub-Agent)。

适合大多数任务: 搜索、分析、写作、生成。

bash:

只有 `bash` 工具。专门用于命令执行。

适合需要大量命令操作的任务：数据处理、代码编译、系统操作。

自定义类型:

可以在 `config.yaml` 定义自定义 Sub-Agent 类型。

```
subagents:
  types:
    - name: research-only
      tools: [web_search, web_fetch, read_file, write_file]
      description: "研究专用，不做执行"
```

08.7 并发限制

为什么限制并发:

- LLM API 有速率限制
- Sandbox 资源有限
- 避免资源耗尽

限制机制:

```
MAX_CONCURRENT_SUBAGENTS = 3
```

通过 `SubagentLimitMiddleware` 强制执行。

如果 Lead Agent 发出4个 `task` 调用，第4个会被截断。

截断逻辑:

```
# SubagentLimitMiddleware.after_model
tool_calls = response.tool_calls
task_calls = [tc for tc in tool_calls if tc.name == "task"]

if len(task_calls) > MAX_CONCURRENT_SUBAGENTS:
    # 截断到3个
    truncated = task_calls[:MAX_CONCURRENT_SUBAGENTS]
    # 替换 response.tool_calls
```

08.8 双线程池

Sub-Agent 执行用两个线程池：

```
_scheduler_pool = ThreadPoolExecutor(max_workers=3) # 调度
_execution_pool = ThreadPoolExecutor(max_workers=3) # 执行
```

调度池：

- 接收任务提交
- 管理任务状态
- 发送 SSE 事件

执行池：

- 实际执行 Sub-Agent
- 处理 LLM 调用
- 工具调用执行

分离的原因：

- 执行可能阻塞（LLM响应慢）
- 调度需要快速响应（推送状态）
- 防止调度阻塞

08.9 超时机制

每个 Sub-Agent 有15分钟超时。

超时后：

- 任务标记为 `task_timed_out`
- 线程池取消执行
- Lead Agent 收到超时通知

15分钟的原因：

- 大多数任务几分钟完成
- 太短会截断长任务
- 太长会占用资源

08.10 独立上下文

关键设计：每个 Sub-Agent 有独立上下文。

它看不到：

- Lead Agent 的对话历史
- 其他 Sub-Agents 的上下文
- 全局 state

只能看到：

- 自己的 prompt
- 自己的工具
- 自己的执行结果

这样设计的意义：

- 聚焦当前任务，不被干扰
- 上下文窗口更干净
- 隐私隔离（不同任务可能涉及不同数据）

08.11 结果汇总

Sub-Agent 返回结构化结果：

```
{
  "status": "completed",
  "result": "收集到以下数据：...",
  "artifacts": ["data.xlsx", "summary.md"]
}
```

Lead Agent 收到结果后：

- 合并到自己的 context
- 可以调用 `read_file` 读取 Sub-Agent 生成的文件
- 继续下一步处理

文件共享：

- Sub-Agent 在同一个 sandbox 执行
- 文件写入共享的 `/mnt/user-data/workspace/`

- Lead Agent 可以直接读取

花叔的经验：Sub-Agent 是处理复杂任务的标准解法。

单 Agent 做复杂任务会爆上下文。Sub-Agent 拆分后，每个只需处理一部分，上下文压力小。汇总时只取结果，不取过程。这才是真正能做完几小时任务的方法。

08.12 向前桥接

Sub-Agents 清楚了。下一节，看 Skills 怎么指导 Agent 做事。

09 Skills：能力模块

09.1 先给结论

Skills 是 DeerFlow 的"工作手册"。

一个 Skill 是一个 Markdown 文件 (`SKILL.md`)。里面定义：

- 怎么做某类任务
- 最佳实践和注意事项
- 参考资源和示例

DeerFlow 内置22个 Skills。你也可以写自己的。

09.2 这节讲什么

这节讲：

1. Skill 是什么
2. 内置 Skills 概览
3. Skill 加载机制
4. 如何自定义 Skill

09.3 Skill 是什么

Skill 是结构化的能力定义。

文件结构：

```
skills/public/deep-research/  
├── SKILL.md           # Skill 定义 (必须有)  
├── references/        # 参考文档 (可选)  
├── assets/            # 资源文件 (可选)  
└── scripts/          # 工具脚本 (可选)
```

[SKILL.md](#) 格式：

```

---
name: deep-research
description: 深度研究流程，从信息收集到报告生成
license: MIT
allowed-tools: [web_search, read_file, write_file, bash]
---

# Deep Research Skill

## 适用场景
- 需要 comprehensive research 的任务
- 多来源信息整合

## 工作流程
1. 确定研究范围
2. 搜索关键信息
3. 整理和分析
4. 生成报告

## 最佳实践
...

```

Frontmatter 字段:

字段	含义	必填
name	Skill 名称	是
description	一句话描述	是
license	许可证	否
allowed-tools	允许的工具	否

09.4 内置 Skills 概览 (22个)

Skill	用途	复杂度
deep-research	深度研究	高
github-deep-research	GitHub 项目研究	高
report-generation	报告生成	中
ppt-generation	演示文稿	中
frontend-design	前端设计	高
web-design-guidelines	网页设计指南	低
data-analysis	数据分析	中
chart-visualization	图表可视化	中
image-generation	图像生成	中
video-generation	视频生成	高
podcast-generation	播客生成	高
newsletter-generation	新闻稿生成	中
academic-paper-review	学术论文评审	高
code-documentation	代码文档生成	低
consulting-analysis	咨询分析	高
skill-creator	Skill 创建辅助	中
find-skills	Skill 发现和推荐	低
bootstrap	项目初始化	中
vercel-deploy-claimable	Vercel 部署	中
surprise-me	随机任务生成	低

Skill	用途	复杂度
claude-to-deerflow	Claude Code 集成	低

高频使用：

- deep-research：研究类任务标配
- report-generation：报告类任务标配
- ppt-generation：演示文稿标配
- frontend-design：网页/UI设计标配

09.5 Skill 加载机制

扫描：

```
load_skills(  
    public_path="deer-flow/skills/public",  
    custom_path="deer-flow/skills/custom"  
)
```

扫描 `skills/public` 和 `skills/custom` 目录，找所有 `SKILL.md`。

解析：

读取 frontmatter，提取：

- name
- description
- allowed-tools
- license

启用状态：

从 `extensions_config.json` 读取启用状态：

```
{
  "skills": {
    "deep-research": {"enabled": true},
    "ppt-generation": {"enabled": true},
    "frontend-design": {"enabled": false}
  }
}
```

注入:

启用的 Skills 写入 system prompt:

```
<skills>
以下 skills 可用:
- /mnt/skills/public/deep-research/SKILL.md
- /mnt/skills/public/ppt-generation/SKILL.md

加载方式: 用 read_file 工具读取。
</skills>
```

09.6 按需加载

Skills 不是一次性全加载。

Agent 在需要时才 `read_file(SKILL.md)`。

这样做的原因:

- 22个 Skills 内容很多, 全加载会爆上下文
- 很多任务只需要1-2个 Skills
- 按需加载节省 token

加载流程:

```
Agent 收到任务: "做一份研究报告"
↓
Agent 判断: 需要 deep-research Skill
↓
Agent 调用: read_file("/mnt/skills/public/deep-research/SKILL.md")
↓
Skill 内容进入 Agent 上下文
↓
Agent 按 Skill 流程执行
```

09.7 自定义 Skill

位置:

`skills/custom/your-skill/SKILL.md`

结构:

```
---
name: my-custom-skill
description: 我的自定义流程
---

# My Custom Skill

## 适用场景
...

## 工作流程
1. 步骤1
2. 步骤2
3. 步骤3

## 注意事项
...
```

启用:

修改 `extensions_config.json`:

```
{
  "skills": {
    "my-custom-skill": {"enabled": true}
  }
}
```

或通过 Gateway API:

```
curl -X PUT http://localhost:2026/api/skills/my-custom-skill \
-H "Content-Type: application/json" \
-d '{"enabled": true}'
```

09.8 Skill 安装

可以从 `.skill` 压缩包安装:

```
curl -X POST http://localhost:2026/api/skills/install \
-F "file=@my-skill.skill"
```

`.skill` 文件是 ZIP 压缩包, 包含:

- `SKILL.md`
- 可选的 `references/`、`assets/`、`scripts/`

安装后解压到 `skills/custom/`。

09.9 Skill 与 Tool 的区别

维度	Skill	Tool
形式	Markdown 文件	Python 函数
内容	流程、最佳实践	原子操作
加载	按需读取	启动时加载
调用	Agent 自己判断	显式工具调用
可扩展	写文件即可	需要写代码

Skill: 告诉 Agent “怎么做”。

Tool: 让 Agent “能做什么”。

两者互补。Skill 指导, Tool 执行。

09.10 Skills 目录结构（容器内）

```
/mnt/skills/
├── public/                # 内置 Skills（只读）
│   ├── deep-research/
│   ├── ppt-generation/
│   ├── frontend-design/
│   └── ...
└── custom/               # 自定义 Skills（可写）
    └── my-custom-skill/
```

Agent 在 sandbox 里看到的是这个路径。

真实路径：

- public: `deer-flow/skills/public/`（宿主机）
- custom: `deer-flow/skills/custom/`（宿主机）

Docker 模式时，这两个目录挂载到容器。

花叔的经验：Skills 是 Agent 的“方法论”。

Tool 给 Agent “能力”，Skill 给 Agent “方法”。光有能力不知道怎么用，任务照样做不好。DeerFlow 的 Skills 设计是我见过最实用的 Agent 能力扩展方案。

09.11 向前桥接

Skills 清楚了。下一节，看 Memory 怎么让 Agent 记住你。

10 Memory System: 长期记忆

10.1 先给结论

Memory System 让 Agent 跨 session 记住你。

记什么：

- 你的偏好（喜欢图表、短句风格）
- 你的背景（技术栈、工作领域）
- 你的习惯（常用 workflows）

存哪：本地文件 `backend/.deer-flow/memory.json`

怎么用：自动注入到 system prompt, Agent 每次对话都能看到。

10.2 这节讲什么

这节讲：

1. Memory 的数据结构
2. Memory 提取机制
3. Memory 注入方式
4. Memory 配置

10.3 Memory 数据结构

```
{
  "workContext": "用户是数据分析师, 常用 Python 和 LangChain",
  "personalContext": "偏好用图表呈现数据, 写作风格偏短句",
  "topOfMind": "最近在做 Agent 相关研究和实践",

  "recentMonths": [
    {"summary": "完成了3次深度研究报告", "date": "2026-03"},
    {"summary": "生成了2份演示文稿", "date": "2026-02"}
  ],

  "facts": [
    {
      "id": "fact-001",
      "content": "用户偏好用图表呈现数据",
      "category": "preference",
      "confidence": 0.9,
      "createdAt": "2026-03-01",
      "source": "conversation"
    },
    {
      "id": "fact-002",
      "content": "用户常用技术栈是 Python、LangChain",
      "category": "knowledge",
      "confidence": 0.8,
      "createdAt": "2026-03-15",
      "source": "conversation"
    }
  ]
}
```

Context 部分:

字段	含义	长度
workContext	工作背景	1-3句
personalContext	个人偏好	1-3句
topOfMind	当前关注	1-3句

History 部分:

字段	含义
recentMonths	近期活动摘要
earlierContext	更早的背景
longTermBackground	长期沉淀

Facts 部分:

每个 fact 有:

- id: 唯一标识
- content: 具体内容
- category: 类别 (preference/knowledge/context/behavior/goal)
- confidence: 置信度 (0-1)
- createdAt: 创建时间
- source: 来源 (conversation)

10.4 Memory 提取机制

触发时机:

每次对话结束后, MemoryMiddleware 推入更新队列。

过滤:

只保留:

- 用户输入
- Agent 最终响应

中间的工具调用、Sub-Agent 交互不参与提取。

队列处理:

```
# MemoryQueue
debounce_seconds = 30 # 等待30秒
max_batch_size = 5    # 批量处理
```

等待30秒后, 批量提交给 LLM 提取。

LLM 提取:

用专门的模型（可配置）分析对话，提取：

- 新的 context 更新
- 新的 facts
- 需要更新的 facts

去重:

新 fact 的 content 与已有 fact 比较（去除空白后），重复则跳过。

原子写入:

```
# 先写临时文件
with open(temp_path, "w") as f:
    f.write(json.dumps(memory_data))

# 再重命名
os.rename(temp_path, final_path)
```

防止写入过程中崩溃导致数据损坏。

10.5 Memory 注入方式

注入时机:

每次 Agent 创建时，从 memory.json 读取，注入到 system prompt。

注入内容:

```
<memory>
用户背景:
- 数据分析师, 常用 Python 和 LangChain
- 偏好用图表呈现数据
- 写作风格偏短句

近期活动:
- 完成了3次深度研究报告
- 生成了2份演示文稿

关键偏好:
- 喜欢用图表呈现数据
- 常用技术栈是 Python、LangChain
</memory>
```

注入限制:

```
memory:
  max_injection_tokens: 2000 # 最多2000 tokens
  max_facts: 100           # 最多100条 facts
  fact_confidence_threshold: 0.7 # 只注入置信度>0.7的
```

超过限制时:

- 只注入置信度最高的 facts
- 只注入最近的 history
- 压缩 context 文本

10.6 Memory 配置

```
# config.yaml
memory:
  enabled: true           # 启用 Memory 系统
  injection_enabled: true # 启用注入到 prompt
  storage_path: backend/.deer-flow/memory.json
  debounce_seconds: 30    # 更新等待时间
  model_name: null        # 提取用的模型 (null=默认模型)
  max_facts: 100          # 最大 facts 数
  fact_confidence_threshold: 0.7 # 置信度阈值
  max_injection_tokens: 2000 # 注入 token 限制
```

10.7 Memory API

查询 Memory:

```
curl http://localhost:2026/api/memory
```

返回完整的 memory.json 内容。

强制刷新:

```
curl -X POST http://localhost:2026/api/memory/reload
```

重新从文件读取，刷新缓存。

查询配置:

```
curl http://localhost:2026/api/memory/config  
curl http://localhost:2026/api/memory/status
```

返回配置和当前状态。

10.8 Memory 与 Privacy

Memory 存在本地文件。你控制数据。

可以:

- 直接编辑 memory.json
- 删除 memory.json 清空记忆
- 通过 API 查看、管理

不会:

- 上传到云端
- 共享给其他用户
- 被 DeerFlow 团队访问

10.9 Memory 的局限

局限1：依赖对话质量

如果对话没有透露偏好、背景，Memory 无法提取。

局限2：置信度机制

低置信度的 facts 不注入。可能遗漏有用信息。

局限3：容量限制

超过100条 facts 或2000 tokens 时截断。早期信息可能丢失。

花叔的经验：Memory 是 Agent 从"工具"变"助手"的关键。

没有 Memory 的 Agent 每次都是新开始。有 Memory 的 Agent 能记住你的风格、习惯、背景。这才是"长期协作"的基础。DeerFlow 的 Memory 设计简单但实用，关键是本地存储、用户可控。

10.10 向前桥接

Memory 清楚了。下一节，看配置文件怎么控制整个系统。

11 配置详解：config.yaml 与 extensions_config.json

11.1 先给结论

DeerFlow 用两个配置文件：

- **config.yaml**：主配置（模型、工具、sandbox、memory等）
- **extensions_config.json**：扩展配置（MCP Server、Skills启用）

配置优先级：

1. 显式参数
2. 环境变量
3. 文件默认值

11.2 这节讲什么

这节讲：

1. config.yaml 结构
2. 模型配置详解
3. sandbox 配置详解
4. extensions_config.json 结构
5. 配置技巧

11.3 config.yaml 结构

```
config_version: 2                # 配置版本号

models:                          # 模型列表
  - name: gpt-4
  ...

tools:                           # 工具配置
  - use: deerflow.community.tavily:tavily_search
  ...

tool_groups:                     # 工具分组
  - name: sandbox
    tools: [bash, ls, read_file, write_file, str_replace]

sandbox:                         # Sandbox 配置
  use: deerflow.community.aio_sandbox:AioSandboxProvider
  ...

skills:                          # Skills 配置
  path: deer-flow/skills
  container_path: /mnt/skills

title:                           # 标题生成配置
  enabled: true
  ...

summarization:                   # 上下文压缩配置
  enabled: true
  ...

subagents:                       # Sub-Agent 配置
  enabled: true
  ...

memory:                          # Memory 配置
  enabled: true
  ...

channels:                        # IM 渠道配置
  feishu:
    enabled: true
  ...
```

11.4 模型配置详解

```
models:
  # 基础配置
  - name: gpt-4                                # 内部标识
    display_name: GPT-4                        # 显示名称
    use: langchain_openai:ChatOpenAI          # LangChain 类路径
    model: gpt-4                                # API 模型标识
    api_key: $OPENAI_API_KEY                   # API Key (环境变量)
    max_tokens: 4096                           # 最大 tokens
    temperature: 0.7                           # 温度

  # 推理模型配置
  - name: doubao-seed
    display_name: Doubao Seed 2.0 Code
    use: langchain_openai:ChatOpenAI
    model: doubao-seed-2-0-code
    api_key: $DOUBAO_API_KEY
    base_url: https://ark.cn-beijing.volces.com/api/v3
    supports_thinking: true                    # 支持扩展思考
    supports_vision: true                     # 支持图像理解
    when_thinking_enabled:                    # thinking 模式时的额外配置
      extra_body:
        chat_template_kwargs:
          enable_thinking: true

  # vLLM 推理模型配置
  - name: qwen-reasoning
    display_name: Qwen Reasoning (vLLM)
    use: deerflow.models.vllm_provider:VllmChatModel
    model: Qwen/Qwen2.5-32B-Instruct
    base_url: http://localhost:8000/v1
    supports_thinking: true
```

字段说明:

字段	含义	必填
name	内部标识符	是
display_name	界面显示名	是
use	LangChain 类路径	是
model	API 模型标识	是
api_key	API Key（可环境变量）	是
base_url	自定义 API 地址	否
max_tokens	最大 tokens	否
temperature	温度	否
supports_thinking	支持扩展思考	否
supports_vision	支持图像理解	否

环境变量：

`$OPENAI_API_KEY` 表示从环境变量读取。

在 `.env` 文件设置：

```
OPENAI_API_KEY=sk-xxx
```

use 字段：

支持的类路径：

- `langchain_openai:ChatOpenAI` - OpenAI 及兼容 API
- `langchain_google_genai:ChatGoogleGenerativeAI` - Google Gemini
- `langchain_anthropic:ChatAnthropic` - Anthropic Claude
- `deerflow.models.vllm_provider:VllmChatModel` - vLLM 自定义

11.5 Sandbox 配置详解

```
sandbox:
  # Local 模式
  use: deerflow.sandbox.local:LocalSandboxProvider

  # Docker 模式 (默认)
  use: deerflow.community.aio_sandbox:AioSandboxProvider

  # Kubernetes 模式
  use: deerflow.community.aio_sandbox:AioSandboxProvider
  provisioner_url: http://provisioner:8002
```

AioSandboxProvider 参数:

```
sandbox:
  use: deerflow.community.aio_sandbox:AioSandboxProvider
  image: deerflow/sandbox:latest      # Sandbox 镜像
  timeout: 900                        # 超时 (秒)
  max_containers: 10                  # 最大容器数
```

11.6 extensions_config.json 结构

```
{
  "mcpServers": {
    "web-search": {
      "enabled": true,
      "type": "stdio",
      "command": "npx",
      "args": [ "-y", "@anthropic/web-search-mcp" ],
      "env": {
        "TAVILY_API_KEY": "$TAVILY_API_KEY"
      },
      "description": "Web search via Tavily"
    },

    "my-http-server": {
      "enabled": true,
      "type": "http",
      "url": "http://my-server:8000/mcp",
      "headers": {
        "Authorization": "Bearer $MY_TOKEN"
      },
      "oauth": {
        "token_url": "https://auth.example.com/token",
        "grant_type": "client_credentials",
        "client_id": "$CLIENT_ID",
        "client_secret": "$CLIENT_SECRET"
      }
    }
  },

  "skills": {
    "deep-research": {"enabled": true},
    "ppt-generation": {"enabled": true},
    "frontend-design": {"enabled": false}
  }
}
```

MCP Server 类型:

类型	配置方式
stdio	command + args + env
sse	url + headers
http	url + headers + oauth

OAuth 配置:

支持两种 grant type:

- `client_credentials` - 客户端凭证
- `refresh_token` - 刷新令牌

11.7 配置优先级

`config.yaml` 路径:

优先级:

1. `config_path` 参数
2. `DEER_FLOW_CONFIG_PATH` 环境变量
3. `config.yaml` (当前目录)
4. `config.yaml` (父目录, 推荐)

`extensions_config.json` 路径:

优先级:

1. `config_path` 参数
2. `DEER_FLOW_EXTENSIONS_CONFIG_PATH` 环境变量
3. `extensions_config.json` (当前目录)
4. `extensions_config.json` (父目录, 推荐)

11.8 配置版本管理

`config.yaml` 有 `config_version` 字段。

当 DeerFlow 更新配置格式时, 版本号会上升。

启动时检查:

- 用户版本 vs 示例版本
- 版本过低则警告

升级配置：

```
make config-upgrade
```

自动合并缺失字段，保留用户配置。

11.9 配置热更新

Gateway 和 LangGraph 都有配置缓存。

但会自动检测文件修改（mtime）：

```
# 检测逻辑
if file_mtime > cached_mtime:
    reload_config()
```

修改 config.yaml 后，重启服务，或等待自动检测刷新。

11.10 常用配置技巧

技巧1：多模型配置

```
models:
  - name: fast                # 快速模型
    use: langchain_openai:ChatOpenAI
    model: gpt-4o-mini

  - name: smart               # 智能模型
    use: langchain_openai:ChatOpenAI
    model: gpt-4
    supports_thinking: true

  - name: vision              # 图像模型
    use: langchain_openai:ChatOpenAI
    model: gpt-4-vision-preview
    supports_vision: true
```

运行时选择：

```
config.configurable.model_name = "smart"
```

技巧2: 环境变量集中管理

`.env` 文件:

```
OPENAI_API_KEY=sk-xxx  
TAVILY_API_KEY=tvly-xxx  
DOUBAO_API_KEY=xxx
```

config.yaml 用 `$VAR_NAME` 引用。

技巧3: Skills 按场景启用

日常研究:

```
{"skills": {"deep-research": true, "report-generation": true}}
```

做演示:

```
{"skills": {"ppt-generation": true, "image-generation": true}}
```

花叔的经验: 配置文件是系统的"驾驶舱"。

DeerFlow 的配置项很多, 但结构清晰。关键是理解每个配置块的作用。我建议先用默认配置跑通, 再根据实际需求逐项调整。不要一开始就改太多, 容易出问题。

11.11 向前桥接

配置清楚了。下一Part, 讲怎么部署 DeerFlow。

Part 3: 部署方案

这Part讲怎么把 DeerFlow 跑起来。从本地到生产。

12 本地开发部署

12.1 先给结论

本地开发部署用 `make dev`。

启动4个进程：LangGraph、Gateway、Frontend、Nginx。

访问 <http://localhost:2026>。

适合：开发调试、功能测试、快速迭代。

12.2 这节讲什么

这节讲：

1. 本地部署的完整流程
2. 各进程的启动和停止
3. 开发调试技巧
4. 常见问题

12.3 环境准备

必须安装：

工具	版本	用途
Python	3.12+	后端运行
Node.js	22+	前端运行
uv	最新	Python 包管理
pnpm	最新	Node 包管理
nginx	最新	反向代理

检查依赖：

```
make check
```

预期输出：

```
✓ Python 3.12.x
✓ Node.js 22.x
✓ uv 0.x.x
✓ pnpm 9.x.x
✓ nginx 1.x.x
```

12.4 安装流程

第一步：克隆仓库

```
git clone https://github.com/bytedance/deer-flow.git
cd deer-flow
```

第二步：生成配置

```
make config
```

生成：

- `config.yaml`
- `.env`
- `extensions_config.json`

第三步：配置模型

编辑 `config.yaml`，至少一个模型：

```
models:
  - name: gpt-4
    display_name: GPT-4
    use: langchain_openai:ChatOpenAI
    model: gpt-4
    api_key: $OPENAI_API_KEY
```

第四步：设置 API Key

编辑 `.env`：

```
OPENAI_API_KEY=sk-your-key  
TAVILY_API_KEY=tvly-your-key
```

第五步：安装依赖

```
make install
```

安装：

- backend 依赖 (uv)
- frontend 依赖 (pnpm)

耗时约2-5分钟。

第六步：启动服务

```
make dev
```

启动顺序：

1. LangGraph Server (2024)
2. Gateway API (8001)
3. Frontend (3000)
4. Nginx (2026)

12.5 进程管理

查看进程：

```
ps aux | grep -E 'langgraph|gateway|next|nginx'
```

停止服务：

```
make stop
```

停止所有4个进程。

重启服务：

```
make dev # 再次执行
```

单独启动：

```
# 后端目录
cd backend

make dev # 只启动 LangGraph
make gateway # 只启动 Gateway
```

12.6 开发调试

日志查看：

每个进程有自己的日志：

进程	日志位置
LangGraph	stdout
Gateway	stdout
Frontend	stdout
Nginx	<code>/var/log/nginx/</code>

端口测试：

```
curl http://localhost:2026/health
curl http://localhost:8001/health
curl http://localhost:2024/info
```

前端开发模式：

前端支持热更新。修改 `frontend/src/` 下的文件，自动刷新。

后端开发模式：

后端需要重启。修改 `backend/` 下的代码后：

```
make stop
make dev
```

12.7 Windows 用户注意

Windows 需要用 Git Bash。

不能直接在 `cmd.exe` 或 `PowerShell` 执行。

原因：脚本依赖 `bash` 和 `Git for Windows` 的 `cygpath` 等工具。

WSL 也不保证可用。

建议：Windows 用户用 `Docker` 方式部署。

12.8 常见问题

Q：make check 报错找不到 uv？

A：安装 uv：

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Q：make install 报错网络超时？

A：国内网络问题。尝试：

- 设置代理
- 用国内镜像源

Q：make dev 启动后访问不了？

A：检查：

- 4个进程是否都启动

- 端口是否被占用
- config.yaml 是否正确

Q：修改配置后不生效？

A：重启服务，或等待自动检测刷新。

花叔的经验：本地部署的坑在配置文件。

make dev 本身很顺。问题通常出在 config.yaml 配置不对。我建议首次部署用最简单的配置：一个 GPT-4 模型，其他都用默认。跑通了再加其他模型和功能。

12.9 向前桥接

本地开发部署清楚了。下一节，看 Docker 部署。

13 Docker 部署（推荐）

13.1 先给结论

Docker 部署是推荐方式。

一条命令启动：`make docker-start`。

优点：

- 不需要安装 Python、Node.js、nginx
- Sandbox 自动用 Docker 模式
- 镜像构建一次，后续秒启动

适合：生产环境、长期运行、多用户共享。

13.2 这节讲什么

这节讲：

1. Docker 部署流程
2. 开发模式 vs 生产模式
3. 数据持久化
4. 常见问题

13.3 前提条件

必须安装：

- Docker（最新版）
- Docker Compose（或 Docker Desktop）

检查：

```
docker --version
docker compose version
```


13.4 开发模式部署

第一步：克隆仓库

```
git clone https://github.com/bytedance/deer-flow.git
cd deer-flow
```

第二步：生成配置

```
make config
```

第三步：配置模型和 API Key（同本地部署）

第四步：拉取 Sandbox 镜像

```
make docker-init
```

拉取 DeerFlow 的 sandbox 基础镜像。

首次执行耗时约2-3分钟。

第五步：启动服务

```
make docker-start
```

启动：

- langgraph 容器
- gateway 容器
- frontend 容器
- nginx 容器
- sandbox 容器（按需创建）

第六步：访问

<http://localhost:2026>

13.5 开发模式特点

特点	说明
源码挂载	修改代码自动生效（前端热更新）
端口映射	所有端口暴露到宿主机
日志输出	stdout 可见
Sandbox	动态创建容器

源码挂载：

容器内路径		宿主机路径
/app/backend	→	deer-flow/backend/
/app/frontend	→	deer-flow/frontend/
/mnt/skills	→	deer-flow/skills/

修改宿主机代码，容器内自动同步。

13.6 生产模式部署

第一步：构建镜像

```
make up
```

构建：

- backend 镜像
- frontend 镜像
- nginx 镜像

耗时约5-10分钟。

第二步：启动服务

```
make up
```

启动全部生产容器。

第三步：停止服务

```
make down
```

停止并移除容器。

13.7 生产模式特点

特点	说明
镜像构建	代码打包进镜像，不挂载源码
端口限制	只有 nginx 2026 暴露
日志管理	Docker 日志驱动
Sandbox	预创建池或动态创建

13.8 数据持久化

Docker Volume:

Volume	内容
deer-flow-data	backend/.deer-flow/
deer-flow-skills	skills/

持久化目录:

```
backend/.deer-flow/  
├── threads/{thread_id}/ # Thread 数据  
├── memory.json          # Memory 数据  
└── checkpoints/         # LangGraph checkpoint
```

Volume 管理:

```
# 查看 volumes
docker volume ls

# 查看具体 volume
docker volume inspect deer-flow-data

# 清理 (危险)
docker volume rm deer-flow-data
```

13.9 Sandbox 容器管理

开发模式：

每个 thread 动态创建 sandbox 容器。

容器命名： `deerflow-sandbox-{thread_id}`

生产模式：

可以配置容器池，避免频繁创建销毁。

```
sandbox:
  pool_size: 5
  max_containers: 20
```

查看 sandbox 容器：

```
docker ps | grep sandbox
```

清理 sandbox 容器：

```
docker rm -f $(docker ps -aq -f name=deerflow-sandbox)
```

13.10 配置文件位置

Docker 模式下，配置文件从宿主机挂载：

容器内路径	宿主机路径
/app/config.yaml	→ deer-flow/config.yaml
/app/.env	→ deer-flow/.env

修改宿主机配置，重启容器生效。

13.11 常见问题

Q: make docker-init 拉镜像超时?

A: 国内网络问题。尝试:

- 配置 Docker 代理
- 用国内镜像源
- 手动拉取: `docker pull deerflow/sandbox:latest`

Q: make docker-start 启动失败?

A: 检查:

- Docker 是否运行
- 端口是否被占用 (2026、2024、8001、3000)
- config.yaml 是否正确

Q: 容器内看不到修改的代码?

A: 开发模式会挂载源码。确保在项目根目录执行 `make docker-start`。

Q: 数据丢失了?

A: 检查 volume 是否正确挂载。不要用 `docker compose down -v` (会删除 volumes)。

花叔的经验: Docker 部署比本地部署更稳。

主要坑在网络拉镜像。国内用户建议先手动拉好基础镜像，再执行 `make docker-start`。首次部署耗时可能10分钟，后续秒级启动。

13.12 向前桥接

Docker 部署清楚了。下一节，看生产环境部署的注意事项。

14 生产环境部署

14.1 先给结论

生产环境部署的关键：

1. 安全配置（访问控制）
2. 资源规划（CPU、内存、存储）
3. 监控告警（LangSmith、日志）
4. 备份策略（数据持久化）

DeerFlow 默认设计为本地可信环境。公网部署必须加安全措施。

14.2 这节讲什么

这节讲：

1. 安全配置
2. 资源规划
3. 监控方案
4. 备份策略
5. 运维检查清单

14.3 安全配置

风险提示：

DeerFlow 具备：

- 系统命令执行（bash）
- 文件读写操作
- 业务逻辑调用

如果暴露到公网，可能导致：

- 未授权执行命令
- 数据泄露

- 法律合规风险

安全措施:

措施	方法
IP 白名单	iptables 或防火墙
身份验证	nginx 反向代理 + auth
网络隔离	VLAN 或独立网段
HTTPS	nginx SSL 配置

IP 白名单:

```
# iptables 示例
iptables -A INPUT -p tcp --dport 2026 -s 10.0.0.1 -j ACCEPT
iptables -A INPUT -p tcp --dport 2026 -j DROP
```

只允许 10.0.0.1 访问 2026 端口。

nginx 身份验证:

```
server {
    listen 2026;

    auth_basic "DeerFlow";
    auth_basic_user_file /etc/nginx/.htpasswd;

    location / {
        proxy_pass http://frontend:3000;
    }
}
```

创建密码文件:

```
htpasswd -c /etc/nginx/.htpasswd admin
```

14.4 资源规划

场景	CPU	内存	存储	说明
单用户体验	4核	8GB	20GB	最小配置
小团队（5人）	8核	16GB	40GB	共享服务
中团队（20人）	16核	32GB	100GB	多会话并发
大团队（50人+）	32核	64GB	200GB	需考虑 K8s

存储增长预估：

数据类型	单 thread 占用	100 threads
workspace	10-100MB	1-10GB
uploads	用户上传量	不确定
outputs	10-50MB	1-5GB
checkpoints	1-5MB	100-500MB
memory.json	<1MB	<1MB

建议预留 50% 空间缓冲。

14.5 监控方案

LangSmith 链路追踪：

```
# .env
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2_xxx
LANGSMITH_PROJECT=deerflow-prod
```

启用后，所有 LLM 调用、Agent 执行、工具调用被追踪。

访问 smith.langchain.com 查看。

日志收集:

```
# docker-compose.yml
services:
  gateway:
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

Docker 日志自动收集, 限制大小。

健康检查:

```
curl http://localhost:2026/health
```

返回:

```
{"status": "healthy"}
```

建议配置定时脚本检查健康状态。

14.6 备份策略

备份内容:

数据	位置	备份方式
Thread 数据	backend/.deer-flow/threads/	定时备份
Memory	backend/.deer-flow/memory.json	定时备份
配置	config.yaml, .env, extensions_config.json	手动备份
Checkpoints	backend/.deer-flow/checkpoints/	可不备份

备份脚本:

```
#!/bin/bash
DATE=$(date +%Y%m%d)
tar -czf deerflow-backup-$DATE.tar.gz \
    backend/.deer-flow/threads \
    backend/.deer-flow/memory.json \
    config.yaml .env extensions_config.json
```

恢复流程:

```
# 停止服务
make down

# 解压备份
tar -xzf deerflow-backup-20260401.tar.gz

# 启动服务
make up
```

14.7 运维检查清单

部署前检查:

- ☐ Docker 正常运行
- ☐ 端口不被占用
- ☐ config.yaml 配置正确
- ☐ API Key 已设置
- ☐ 资源足够 (CPU、内存、存储)
- ☐ 安全措施已配置

部署后检查:

- ☐ 所有容器正常运行
- ☐ 健康检查返回 healthy
- ☐ 能正常发消息
- ☐ Sandbox 能执行命令
- ☐ 文件能上传和读取

日常运维检查:

- ☐ 日志无异常

- [] 存储空间充足
- [] LangSmith 追踪正常
- [] 备份定时执行
- [] 响应时间正常

14.8 常见问题

Q: 生产环境用 Local Sandbox 安全吗?

A: 不安全。必须用 Docker Sandbox 或 Kubernetes Sandbox。

Q: 公网部署怎么配置 HTTPS?

A: nginx SSL 配置:

```
server {  
    listen 443 ssl;  
    ssl_certificate /path/to/cert.pem;  
    ssl_certificate_key /path/to/key.pem;  
    ...  
}
```

Q: 多用户怎么隔离?

A: DeerFlow 天然按 thread 隔离。不同用户的 thread 数据不共享。

Q: 怎么限制用户只能用某些模型?

A: 通过 Gateway API 控制模型列表。或在前端做权限控制。

花叔的经验: 生产部署的核心是"可控"。

本地体验不管安全, 但生产必须管。IP 白名单、身份验证、HTTPS, 这三项是基本配置。监控和备份是运维刚需, LangSmith 的追踪非常有用。

14.9 向前桥接

生产环境部署清楚了。下一节, 看性能调优。

15 性能调优

15.1 先给结论

性能调优的三个方向：

1. **Agent 执行**：上下文管理、工具调用效率
2. **Sandbox 执行**：容器启动、命令执行
3. **系统资源**：并发控制、内存管理

关键配置：上下文压缩、Sub-Agent 并发限制、Sandbox 容器池。

15.2 这节讲什么

这节讲：

1. 上下文管理调优
2. Sandbox 性能调优
3. 并发控制
4. 资源监控

15.3 上下文管理调优

问题：长对话爆上下文窗口。

解决方案：启用 Summarization。

```
# config.yaml
summarization:
  enabled: true
  trigger_tokens_fraction: 0.7 # 达到70%时触发
  keep_first_n_messages: 5     # 保留前5条
  keep_last_n_messages: 10    # 保留后10条
  model_name: gpt-4o-mini     # 用便宜模型压缩
```

压缩策略：

配置项	含义
trigger_tokens_fraction	达到窗口多少比例时触发
trigger_tokens_absolute	达到绝对 tokens 数触发
keep_first_n_messages	保留开头的N条
keep_last_n_messages	保留结尾的N条

压缩效果：

100条对话 → 压缩后可能只剩：

- 前5条原文
- 中间部分的摘要
- 后10条原文

上下文从 50k tokens → 20k tokens。

15.4 Sandbox 性能调优

问题：Docker 容器启动慢。

每次 thread 创建 sandbox 容器，耗时约 5-10 秒。

解决方案：容器池。

```
# config.yaml (规划中功能)
sandbox:
  pool_size: 5          # 预创建5个容器
  max_containers: 20    # 最大容器数
```

容器池预创建容器，thread 直接从池获取，无需等待启动。

当前替代方案：

如果不需要完全隔离，可以考虑 Local Sandbox：

```
sandbox:
  use: deerflow.sandbox.local:LocalSandboxProvider
```

优点：无容器启动延迟。 缺点：隔离性差，不适合生产。

15.5 Sub-Agent 并发调优

当前限制：最多 3 个并发。

代码中硬编码：

```
MAX_CONCURRENT_SUBAGENTS = 3
```

调优考虑：

- LLM API 速率限制
- Sandbox 资源
- 上下文管理

建议：

场景	并发数
单用户体验	3（默认）
生产环境	3（避免超限）
高 API 限额	可调高（修改代码）

如果要调整并发数，修改：

```
# packages/harness/deerflow/subagents/executor.py
MAX_CONCURRENT_SUBAGENTS = 5 # 改成5
```

同时调整线程池：

```
_scheduler_pool = ThreadPoolExecutor(max_workers=5)
_execution_pool = ThreadPoolExecutor(max_workers=5)
```

15.6 线程池调优

Sub-Agent 执行用双线程池。

```
_scheduler_pool = ThreadPoolExecutor(max_workers=3)
_execution_pool = ThreadPoolExecutor(max_workers=3)
```

调优建议：

资源	线程池大小
4核 CPU	3（默认）
8核 CPU	5-6
16核 CPU	8-10

调整时同时调整 MAX_CONCURRENT_SUBAGENTS。

15.7 Memory 调优

问题：Memory 提取占用 tokens。

调优：

```
memory:
  debounce_seconds: 30    # 等待时间
  max_injection_tokens: 2000 # 注入限制
```

debounce_seconds:

等待时间越长，批量越大，LLM 调用越少。

但用户下次对话可能看不到最新 memory。

建议：

- 高频使用：30秒
- 低频使用：60秒

max_injection_tokens:

限制 memory 注入的 tokens。

太大：挤占任务上下文。 太小：memory 信息不全。

建议：1500-2500。

15.8 资源监控

Docker 资源监控：

```
docker stats
```

输出：

CONTAINER	CPU %	MEM USAGE	NET I/O
deerflow-gateway	5%	500MB	10MB
deerflow-frontend	2%	200MB	5MB
deerflow-langgraph	10%	1GB	20MB
deerflow-sandbox-1	15%	300MB	1MB

LangSmith 监控：

查看：

- LLM 调用耗时
- Token 消耗
- 错误率
- Agent 执行链路

日志监控：

关键日志：

- Agent 执行超时
- Sandbox 启动失败
- LLM API 错误
- Memory 更新失败

15.9 性能问题排查

问题1： 响应很慢。

排查：

- 检查 LLM API 响应时间 (LangSmith)
- 检查 Sandbox 容器启动时间
- 检查工具调用数量

问题2： 内存持续增长。

排查：

- 检查 Sandbox 容器数量
- 检查 thread 数据大小
- 启用 Summarization

问题3： Sub-Agent 超时。

排查：

- 检查任务复杂度
- 检查 LLM API 响应
- 调整 max_turns

15.10 性能调优检查清单

- ☐ Summarization 已启用
- ☐ Sandbox 用 Docker 模式 (生产)
- ☐ Sub-Agent 并发数适当
- ☐ Memory 注入 tokens 限制合理
- ☐ 资源监控已配置
- ☐ LangSmith 已启用

花叔的经验： 性能调优的关键是"上下文管理"。

Agent 的大头是 LLM 调用，不是系统开销。上下文爆了，调什么都没用。Summarization 是必须启用的功能，尤其是做长任务。

15.11 向前桥接

性能调优清楚了。下一Part，讲 DeerFlow 的核心功能使用。

Part 4: 核心功能

这Part讲 DeerFlow 能做什么。每节一个实际场景。

16 Web 界面使用

16.1 先给结论

Web 界面是 DeerFlow 的主要交互方式。

功能：

- 发送消息和接收响应
- 选择模型和执行模式
- 上传文件（PDF/PPT/Excel）
- 查看 thread 历史
- 下载输出文件

访问地址：<http://localhost:2026>

16.2 这节讲什么

这节讲：

1. 界面功能一览
2. 执行模式选择
3. Thread 管理
4. 文件操作

16.3 界面功能一览

顶部：

- 模型选择下拉框
- 执行模式切换（Flash/Standard/Pro/Ultra）

左侧：

- Thread 列表
- 新建 Thread 按钮

中间：

- 聊天区域
- 消息输入框
- 文件上传按钮

右侧：

- 输出文件列表
- Thread 信息

16.4 执行模式选择

模式	含义	适用场景
Flash	快速响应，无思考	简单问答
Standard	标准执行	日常任务
Pro	规划模式（Plan Mode）	复杂任务，需拆解
Ultra	Sub-Agent 模式	大任务，需并行

Flash 模式：

最快响应。不做规划，不拆任务。

适合：

- 简单问答
- 快速搜索
- 文件读取

Standard 模式：

标准执行。单 Agent 执行所有步骤。

适合：

- 日常任务
- 中等复杂度
- 不需要并行

Pro 模式：

启用 TodoList。Agent 先规划任务列表，再逐项执行。

适合：

- 复杂任务
- 多步骤任务
- 需要进度追踪

Ultra 模式：

启用 Sub-Agent。大任务拆分并行执行。

适合：

- 研究类任务
- 大数据分析
- 报告生成

16.5 Thread 管理

创建 Thread：

点击左侧"+"按钮，或发送第一条消息自动创建。

Thread 信息：

每个 Thread 显示：

- 标题（自动生成）
- 创建时间
- 最后活跃时间

切换 Thread：

点击左侧 Thread 列表项。

删除 Thread：

点击 Thread 右侧的删除按钮。

删除后：

- LangGraph thread 数据删除
- 本地 thread 目录删除
- Sandbox 容器清理（Docker模式）

16.6 文件操作

上传文件：

点击消息输入框旁边的上传按钮。

支持：

- PDF
- PPT
- Excel
- Word
- 图片
- 其他文本文件

PDF/PPT/Excel/Word 自动转换成 Markdown。

上传后：

文件存入 `/mnt/user-data/uploads/`。

Agent 收到文件列表，可以：

- `read_file` 读取内容
- 分析文件内容
- 基于文件生成输出

下载输出：

Agent 生成的文件存入 `/mnt/user-data/outputs/`。

右侧输出文件列表显示可下载的文件。

点击下载。

16.7 消息发送

发送消息：

输入框输入文本，点击发送或回车。

消息格式：

可以是：

- 纯文本
- 带图片的消息（上传图片后发送）

响应格式：

Agent 响应可以是：

- 纯文本
- Markdown 格式（表格、代码块）
- 文件列表（输出文件）
- 图片（如果生成）

响应流式输出：

响应实时显示，不用等全部生成。

16.8 模型切换

切换模型：

顶部下拉框选择模型。

切换后，当前 thread 的后续对话用新模型。

模型信息：

下拉框显示：

- display_name
- 是否支持 vision（图标）
- 是否支持 thinking（图标）

16.9 执行模式切换

切换模式：

顶部模式按钮切换。

切换后，当前 thread 的后续对话用新模式。

模式参数：

模式	thinking_enabled	is_plan_mode	subagent_enabled
Flash	false	false	false
Standard	true	false	false
Pro	true	true	false
Ultra	true	false	true

花叔的经验：Web 界面的关键是"模式选择"。

不同任务用不同模式。简单问答用 Flash，复杂任务用 Pro 或 Ultra。不要所有任务都用 Standard，会浪费资源。

16.10 向前桥接

Web 界面清楚了。下一节，看 IM 渠道集成。

17 IM 渠道集成

17.1 先给结论

DeerFlow 支持从 IM 应用接收任务。

支持渠道：

- Telegram
- Slack
- 飞书
- 企业微信

配置完成后，直接在聊天窗口和 DeerFlow 交互。

不需要公网 IP。

17.2 这节讲什么

这节讲：

1. IM 渠道架构
2. 各渠道配置方法
3. 命令和交互
4. 常见问题

17.3 IM 渠道架构

组件：

组件	功能
MessageBus	消息路由（入站/出站）
ChannelManager	管理渠道生命周期
Store	映射 chat → thread

流程：

```
IM平台 → Channel实现 → MessageBus → LangGraph → Agent执行
      ↑
      |—— 出站消息 ← MessageBus
```

特点：

- 不需要公网 IP
- 用 long-polling 或 WebSocket
- Thread 自动创建和管理

17.4 Telegram 配置

第一步：创建 Bot

1. 打开 [@BotFather](#)
2. 发送 `/newbot`
3. 按提示创建
4. 复制 HTTP API Token

第二步：配置

```
# config.yaml
channels:
  telegram:
    enabled: true
    bot_token: $TELEGRAM_BOT_TOKEN
    allowed_users: [] # 空表示允许所有人
```

```
# .env
TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklMNOpqrSTUvwxyz
```

第三步：启动

DeerFlow 启动后自动连接 Telegram Bot API。

测试：

在 Telegram 找到你的 Bot，发送消息。

17.5 Slack 配置

第一步：创建 Slack App

1. 前往 api.slack.com/apps
2. Create New App → From scratch
3. 输入 App 名称和工作区

第二步：配置权限

在 OAuth & Permissions 添加 Bot Token Scopes:

- `app_mentions:read`
- `chat:write`
- `im:history`
- `im:read`
- `im:write`
- `files:write`

第三步：启用 Socket Mode

1. 启用 Socket Mode
2. 生成 App-Level Token (`xapp-...`)
3. 权限: `connections:write`

第四步：订阅事件

在 Event Subscriptions 订阅 bot events:

- `app_mention`
- `message.im`

第五步：安装 App

Install to Workspace, 获取 Bot Token (`xoxb-...`)。

第六步：配置

```
# config.yaml
channels:
  slack:
    enabled: true
    bot_token: $SLACK_BOT_TOKEN
    app_token: $SLACK_APP_TOKEN
    allowed_users: []
```

```
# .env
SLACK_BOT_TOKEN=xoxb-...
SLACK_APP_TOKEN=xapp-...
```

17.6 飞书配置

第一步：创建应用

1. 前往 [飞书开放平台](#)
2. 创建企业自建应用
3. 启用 Bot 能力

第二步：添加权限

添加权限：

- `im:message`
- `im:message.p2p_msg:readonly`
- `im:resource`

第三步：配置事件订阅

1. 订阅 `im.message.receive_v1`
2. 连接方式选择"长连接"

第四步：配置

```
# config.yaml
channels:
  feishu:
    enabled: true
    app_id: $FEISHU_APP_ID
    app_secret: $FEISHU_APP_SECRET
    # domain: https://open.feishu.cn # 国内版 (默认)
    # domain: https://open.larksuite.com # 国际版
```

```
# .env
FEISHU_APP_ID=cli_xxxx
FEISHU_APP_SECRET=your_app_secret
```

17.7 企业微信配置

第一步：创建机器人

在企业微信智能机器人平台创建机器人。

获取 `bot_id` 和 `bot_secret`。

第二步：配置

```
# config.yaml
channels:
  wecom:
    enabled: true
    bot_id: $WECOM_BOT_ID
    bot_secret: $WECOM_BOT_SECRET
```

```
# .env
WECOM_BOT_ID=your_bot_id
WECOM_BOT_SECRET=your_bot_secret
```

第三步：重启服务

企业微信渠道通过 WebSocket 连接，无需公网回调地址。

17.8 命令和交互

命令列表：

命令	说明
<code>/new</code>	开启新对话
<code>/status</code>	查看当前 thread 信息
<code>/models</code>	列出可用模型
<code>/memory</code>	查看 memory
<code>/help</code>	查看帮助

普通消息：

没有命令前缀的消息被当作普通聊天。

DeerFlow 自动创建 thread，以对话方式回复。

文件交互：

部分渠道支持：

- 发送图片（Agent 能看到）
- 发送文件（部分渠道支持）
- 接收 Agent 生成的图片/文件

17.9 渠道对比

维度	Telegram	Slack	飞书	企业微信
上手难度	简单	中等	中等	中等
传输方式	long-polling	Socket Mode	WebSocket	WebSocket
公网IP	不需要	不需要	不需要	不需要
图片支持	支持	支持	支持	支持
文件支持	支持	支持	支持	支持

花叔的经验：Telegram 最简单，适合快速体验。

如果只是想试试 IM 渠道，用 Telegram。创建 Bot 只需 1 分钟。Slack 和飞书配置稍复杂，但功能更完善。

17.10 向前桥接

IM 渠道清楚了。下一节，看 Claude Code 集成。

18 Claude Code 集成

18.1 先给结论

通过 `claude-to-deerflow` skill, 可以在 Claude Code 里直接和 DeerFlow 交互。

功能:

- 发送消息给 DeerFlow
- 选择执行模式
- 检查健康状态
- 管理 threads
- 上传文件

不需要离开终端。

18.2 这节讲什么

这节讲:

1. 安装 `claude-to-deerflow` skill
2. 使用方法
3. 功能详解
4. 常见问题

18.3 安装 skill

前提:

- 已安装 Claude Code
- DeerFlow 已启动 (默认 <http://localhost:2026>)

安装命令:

```
npx skills add https://github.com/bytedance/deer-flow --skill claude-to-deerflow
```

安装后, 在 Claude Code 里用 `/claude-to-deerflow` 命令。

18.4 使用方法

启动：

在 Claude Code 里输入：

```
/claude-to-deerflow
```

或简写：

```
/deerflow
```

发送消息：

```
/deerflow 帮我做一份关于新能源汽车行业的研究报告
```

选择模式：

```
/deerflow --mode ultra 帮我做一份深度研究报告
```

18.5 功能详解

发送消息：

```
/deerflow [消息内容]
```

发送消息给 DeerFlow，接收流式响应。

选择模式：

参数	模式
<code>--mode flash</code>	快速模式
<code>--mode standard</code>	标准模式（默认）
<code>--mode pro</code>	规划模式
<code>--mode ultra</code>	Sub-Agent 模式

检查状态：

```
/deerflow --health
```

返回：

- DeerFlow 是否运行
- 可用模型列表
- 可用 skills 列表

管理 threads：

```
/deerflow --threads
```

列出所有 threads。

```
/deerflow --thread <thread_id> --delete
```

删除指定 thread。

上传文件：

```
/deerflow --upload ./report.pdf
```

上传文件到当前 thread。

18.6 环境变量

自定义端点:

```
# .env 或 shell export
DEERFLOW_URL=http://localhost:2026
DEERFLOW_GATEWAY_URL=http://localhost:2026
DEERFLOW_LANGGRAPH_URL=http://localhost:2026/api/langgraph
```

远程 DeerFlow:

```
export DEERFLOW_URL=https://your-deerflow-server.com
```

可以连接远程部署的 DeerFlow。

18.7 使用场景

场景1: 在 Claude Code 里做研究

```
/deerflow --mode ultra 帮我研究一下 LangGraph 和 LangChain 的区别
```

DeerFlow 会启动 Sub-Agents 并行研究。

场景2: 快速问答

```
/deerflow --mode flash 什么是 Agent?
```

快速响应, 不需要思考。

场景3: 复杂任务规划

```
/deerflow --mode pro 帮我设计一个 Agent 系统, 包含以下功能...
```

Pro 模式会先规划任务列表, 再执行。

18.8 与 Claude Code 的协作

Claude Code 本身是 Coding Agent。

DeerFlow 是 Super Agent Harness。

协作方式：

1. Claude Code 处理代码任务
2. DeerFlow 处理研究、报告、生成类任务

分工示例：

```
# 用 Claude Code 写代码
帮我写一个 Python 函数，实现...

# 用 DeerFlow 做研究
/deerflow --mode ultra 帮我研究这个技术方案的可行性

# 用 Claude Code 实现
根据研究结果，帮我实现...
```

18.9 常见问题

Q: /deerflow 命令找不到？

A: 确认 skill 已安装：

```
npx skills list
```

看到 `claude-to-deerflow` 表示已安装。

Q: 连接失败？

A: 检查：

- DeerFlow 是否启动
- DEERFLOW_URL 是否正确
- 端口是否可访问

Q: 响应很慢？

A: 检查模式。Ultra 模式会启动 Sub-Agents，比 Flash 慢。

花叔的经验: Claude Code + DeerFlow 是很实用的组合。

Claude Code 擅长代码, DeerFlow 擅长研究和生成。在终端里同时用两个 Agent, 效率很高。

18.10 向前桥接

Claude Code 集成清楚了。下一节, 看文件上传和文档处理。

19 文件上传和文档处理

19.1 先给结论

DeerFlow 支持文件上传和自动文档转换。

支持格式：

- PDF → Markdown
- PPT → Markdown
- Excel → Markdown
- Word → Markdown
- 图片（直接处理）
- 其他文本文件

转换后存入 sandbox，Agent 可以读取分析。

19.2 这节讲什么

这节讲：

1. 文件上传流程
2. 文档转换机制
3. API 使用方法
4. 常见问题

19.3 文件上传流程

Web 界面：

1. 点击消息输入框旁的上传按钮
2. 选择文件
3. 上传
4. 文件出现在对话中

API：

```
curl -X POST http://localhost:2026/api/threads/{thread_id}/uploads \
-F "files=@report.pdf"
```

上传后:

1. Gateway 接收文件
2. 检测文件类型
3. 如果是 PDF/PPT/Excel/Word, 调用 markdown 转换
4. 存入 `/mnt/user-data/uploads/`
5. 返回文件列表

19.4 文档转换机制

转换工具: markdown

markdown 是微软开源的文档转换工具。

支持:

- PDF → Markdown
- PPT/PPTX → Markdown
- Excel/XLSX → Markdown
- Word/DOCX → Markdown

转换流程:

```
上传文件 → 检测类型 → markdown 转换 → 生成 .md 文件
```

保留原始文件:

同时保留原始文件和转换后的 Markdown。

```
uploads/
├── report.pdf      # 原始文件
└── report.pdf.md  # 转换后的 Markdown
```

Agent 读取:

Agent 收到文件列表, 可以选择:

- 读取转换后的 Markdown (`read_file("report.pdf.md")`)
- 或读取原始文件 (如果是文本格式)

19.5 支持的文件类型

类型	扩展名	处理方式
PDF	.pdf	markdown 转换
PPT	.ppt, .pptx	markdown 转换
Excel	.xls, .xlsx	markdown 转换
Word	.doc, .docx	markdown 转换
图片	.png, .jpg, .gif, .webp	直接存储
文本	.txt, .md, .json, .csv, .yaml	直接存储
代码	.py, .js, .ts, .go, .java 等	直接存储

图片处理:

如果模型支持 vision, Agent 可以"看见"图片。

调用 `view_image` 工具读取图片为 base64。

19.6 API 使用方法

上传文件:

```
POST /api/threads/{thread_id}/uploads
Content-Type: multipart/form-data

files: [File1, File2, ...]
```

响应:

```
{
  "success": true,
  "files": [
    {
      "filename": "report.pdf",
      "size": 102400,
      "converted": true,
      "converted_filename": "report.pdf.md"
    }
  ]
}
```

列出上传文件:

```
GET /api/threads/{thread_id}/uploads/list
```

响应:

```
{
  "files": [
    {
      "filename": "report.pdf",
      "size": 102400,
      "uploaded_at": "2026-04-09T10:00:00Z"
    }
  ],
  "count": 1
}
```

删除文件:

```
DELETE /api/threads/{thread_id}/uploads/{filename}
```

19.7 文件大小限制

默认限制:

配置项	默认值
单文件大小	50MB
总上传大小	200MB

调整限制（修改 Gateway 配置）：

```
# app/gateway/routers/uploads.py
MAX_FILE_SIZE = 100 * 1024 * 1024 # 100MB
MAX_TOTAL_SIZE = 500 * 1024 * 1024 # 500MB
```

19.8 使用场景

场景1：分析 PDF 报告

1. 上传 PDF 报告
2. Agent 读取转换后的 Markdown
3. 分析内容，提取关键信息
4. 生成摘要或回答问题

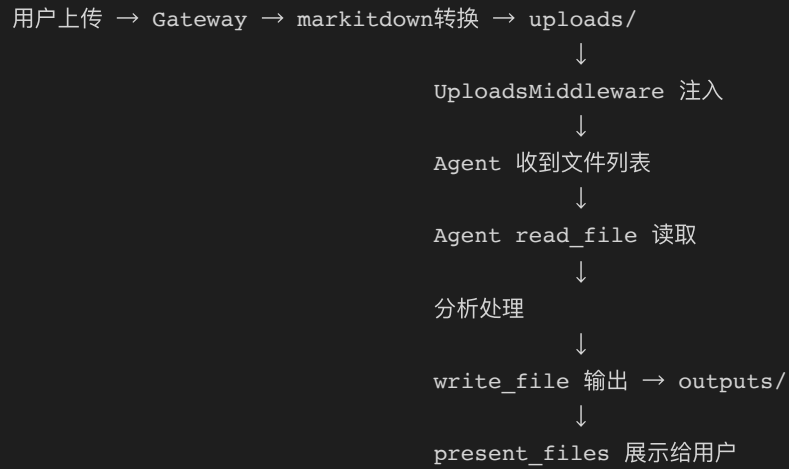
场景2：处理 Excel 数据

1. 上传 Excel 文件
2. Agent 读取数据
3. 分析数据，生成图表
4. 输出分析报告

场景3：理解图片

1. 上传图片
2. Agent 用 `view_image` 查看图片
3. 描述图片内容或回答问题

19.9 文件流转示意



19.10 常见问题

Q: PDF 转换失败?

A: 检查:

- markdown 是否正确安装
- PDF 是否损坏或加密
- 文件大小是否超限

Q: 图片上传后 Agent 看不到?

A: 检查:

- 模型是否支持 vision
- 是否调用了 view_image 工具

Q: 转换后的 Markdown 格式不对?

A: markdown 的转换不完美。复杂格式可能有损失。建议:

- 检查原始 PDF 格式
- 尝试重新转换
- 人工校正

*花叔的经验：*文件上传是 Agent 处理真实任务的基础。

大多数工作任务都涉及文件。DeerFlow 的自动转换很实用，PDF/Excel 直接变成 Markdown，Agent 能直接读。这是我见过最顺手的文件处理方案。

19.11 向前桥接

文件上传清楚了。下一Part，讲 Skills 进阶。

Part 5: Skills 进阶

这Part讲 Skills 的深入用法。从内置到自定义。

20 内置 Skills 概览

20.1 先给结论

DeerFlow 内置 22 个 Skills。

分类：

- 研究类：deep-research、github-deep-research
- 生成类：report-generation、ppt-generation、frontend-design
- 分析类：data-analysis、chart-visualization、consulting-analysis
- 内容类：newsletter-generation、podcast-generation、video-generation
- 其他：skill-creator、find-skills、bootstrap

20.2 这节讲什么

这节讲：

1. 研究类 Skills
2. 生成类 Skills
3. 分析类 Skills
4. 其他常用 Skills

20.3 研究类 Skills

deep-research:

深度研究流程。

工作流：

1. 确定研究范围和关键词
2. 多来源搜索信息
3. 整理和分析
4. 生成结构化报告

适用：

- 行业研究
- 技术调研
- 竞品分析

github-deep-research:

GitHub 项目深度研究。

工作流：

1. 获取项目基本信息
2. 分析代码结构
3. 研究技术栈
4. 生成项目报告

适用：

- 开源项目调研
- 技术选型评估

academic-paper-review:

学术论文评审。

工作流：

1. 读取论文
2. 分析研究方法
3. 评估贡献
4. 生成评审意见

适用：

- 论文评审
- 学术研究

20.4 生成类 Skills

report-generation:

报告生成。

工作流：

1. 确定报告结构
2. 收集素材
3. 撰写内容
4. 格式化输出

适用：

- 研究报告
- 分析报告
- 总结报告

ppt-generation:

演示文稿生成。

工作流：

1. 确定主题和结构
2. 生成大纲
3. 撰写每页内容
4. 输出 HTML（可转 PPT）

适用：

- 演示文稿
- 汇报材料

frontend-design:

前端设计。

工作流：

1. 理解设计需求
2. 设计界面布局
3. 实现代码
4. 输出可运行页面

适用：

- 网页设计
- UI 原型
- Dashboard

image-generation:

图像生成。

工作流:

1. 理解图像需求
2. 调用图像生成 API
3. 处理图像
4. 输出图像文件

适用:

- 配图生成
- 图标设计

video-generation:

视频生成。

工作流:

1. 理解视频需求
2. 生成脚本
3. 调用视频生成 API
4. 输出视频文件

适用:

- 短视频
- 演示视频

podcast-generation:

播客生成。

工作流:

1. 确定主题
2. 撰写脚本
3. 生成音频
4. 输出播客文件

适用:

- 播客内容
- 有声读物

20.5 分析类 Skills

data-analysis:

数据分析。

工作流：

1. 读取数据文件
2. 清洗和处理数据
3. 统计分析
4. 生成分析报告

适用：

- 数据探索
- 业务分析

chart-visualization:

图表可视化。

工作流：

1. 读取数据
2. 选择图表类型
3. 生成图表代码
4. 输出图表

适用：

- 数据可视化
- 报告配图

consulting-analysis:

咨询分析。

工作流：

1. 理解业务问题

2. 分析现状
3. 提出建议
4. 生成咨询报告

适用：

- 业务咨询
- 战略分析

20.6 其他常用 Skills

skill-creator:

创建新 Skill。

工作流：

1. 理解 Skill 需求
2. 设计工作流
3. 编写 [SKILL.md](#)
4. 测试验证

适用：

- 创建自定义 Skill

find-skills:

发现和推荐 Skills。

工作流：

1. 分析任务需求
2. 匹配可用 Skills
3. 推荐最合适的 Skill

适用：

- 找到合适的 Skill

bootstrap:

项目初始化。

工作流：

1. 确定项目类型
2. 生成项目结构
3. 创建基础文件
4. 初始化配置

适用：

- 新项目起步

claude-to-deerflow:

Claude Code 集成（前面已讲）。

web-design-guidelines:

网页设计指南。

提供设计原则和最佳实践。

20.7 Skills 使用建议

场景匹配：

任务类型	推荐 Skills
研究任务	deep-research, github-deep-research
报告撰写	report-generation
演示文稿	ppt-generation
数据分析	data-analysis, chart-visualization
网页开发	frontend-design, web-design-guidelines

组合使用：

多个 Skills 可以组合：

```
deep-research → report-generation → ppt-generation
```

先研究，再写报告，最后做演示文稿。

花叔的经验：内置 Skills 是 DeerFlow 的"即战力"。

不用自己写，直接用。最常用的是 `deep-research` 和 `report-generation`。这两个组合能完成大部分知识工作。

20.8 向前桥接

内置 Skills 清楚了。下一节，看怎么写自定义 Skill。

21 自定义 Skills 开发

21.1 先给结论

自定义 Skill 就是一个目录 + [SKILL.md](#) 文件。

步骤：

1. 创建目录 `skills/custom/my-skill/`
2. 编写 [SKILL.md](#)
3. 启用 Skill
4. 测试验证

不需要写代码，只需要写 Markdown。

21.2 这节讲什么

这节讲：

1. Skill 目录结构
2. [SKILL.md](#) 编写规范
3. Skill 开发流程
4. Skill 测试方法

21.3 Skill 目录结构

基础结构：

```
skills/custom/my-skill/  
└── SKILL.md
```

完整结构：

```
skills/custom/my-skill/
├── SKILL.md           # Skill 定义（必须有）
├── references/        # 参考文档
│   └── best-practices.md
├── assets/           # 资源文件
│   └── template.html
└── scripts/          # 工具脚本
    └── helper.py
```

各目录用途：

目录	用途	必须有
SKILL.md	Skill 定义和工作流	是
references/	参考资料、最佳实践	否
assets/	模板、示例文件	否
scripts/	辅助脚本	否

21.4 [SKILL.md](#) 编写规范

基本格式：


```

---
name: my-custom-skill
description: 一句话描述这个 skill 的功能
license: MIT
allowed-tools: [web_search, read_file, write_file, bash]
---

# My Custom Skill

## 适用场景

描述这个 skill 适合什么任务。

## 工作流程

### 步骤1: XXX

具体说明:
- 做什么
- 怎么做
- 注意什么

### 步骤2: XXX

...

## 最佳实践

1. 建议1
2. 建议2
3. 建议3

## 注意事项

- 注意点1
- 注意点2

## 输出格式

描述最终输出的格式要求。

```

Frontmatter 字段:

字段	类型	必填	说明
name	string	是	Skill 唯一标识
description	string	是	一句话描述
license	string	否	许可证
allowed-tools	array	否	允许使用的工具

name 规则:

- 小写字母
- 用连字符分隔
- 例如: `my-custom-skill`、`data-pipeline`

description 规则:

- 一句话（不超过50字）
- 描述功能，不是用途

allowed-tools 规则:

- 可选配置
- 如果不配置, Agent 可以使用所有工具
- 配置后, Agent 只能用列出的工具

21.5 Skill 开发流程

第一步: 确定 Skill 目标

问自己:

- 这个 Skill 解决什么问题?
- 适用什么场景?
- 需要哪些工具?

第二步: 创建目录

```
mkdir -p skills/custom/my-skill
```

第三步：编写 [SKILL.md](#)

从模板开始：

```
---
name: my-skill
description: 简短描述
---

# My Skill

## 适用场景
...

## 工作流程
...
```

第四步：添加参考文档（可选）

```
mkdir skills/custom/my-skill/references
```

放入参考资料。

第五步：启用 Skill

修改 `extensions_config.json`：

```
{
  "skills": {
    "my-skill": {"enabled": true}
  }
}
```

或通过 API：

```
curl -X PUT http://localhost:2026/api/skills/my-skill \
  -H "Content-Type: application/json" \
  -d '{"enabled": true}'
```

第六步：测试

在 DeerFlow 里发送消息，测试 Skill 是否正常工作。

21.6 Skill 示例：周报生成

```
---
name: weekly-report
description: 根据本周工作内容生成周报
allowed-tools: [read_file, write_file]
---
```

```
# Weekly Report Skill
```

```
## 适用场景
```

每周五下午，整理本周工作，生成周报。

```
## 工作流程
```

```
### 步骤1：收集工作内容
```

询问用户本周做了什么：

- 完成任务
- 进行中的任务
- 遇到的问题

```
### 步骤2：整理分类
```

按以下分类整理：

- 已完成
- 进行中
- 待处理

```
### 步骤3：生成周报
```

按以下格式生成：

```
```markdown
周报（YYYY-MM-DD）
```

```
本周完成
```

- [任务1]
- [任务2]

```
进行中
```

- [任务3]（进度 x%）

```
下周计划
```

- [任务4]

```
- [任务5]
```

```
问题与风险
```

```
- [问题描述]
```

## 步骤4：输出

保存到 `/mnt/user-data/outputs/周报-{date}.md`

## 注意事项

---

- 周报要简洁
- 重点突出成果
- 问题要具体

### ### 21.7 Skill 测试方法

#### **\*\*手动测试\*\*:**

1. 启用 Skill
2. 发送消息触发
3. 检查输出

#### **\*\*测试检查清单\*\*:**

- [ ] Skill 是否被识别
- [ ] 工作流是否按预期执行
- [ ] 输出格式是否正确
- [ ] 是否有遗漏或错误

#### **\*\*调试技巧\*\*:**

如果 Skill 不工作:

1. 检查 SKILL.md 格式是否正确
2. 检查 name 是否唯一
3. 检查 allowed-tools 是否限制
4. 查看 Agent 日志

### ### 21.8 Skill 发布

#### **\*\*打包\*\*:**

```
```bash
cd skills/custom/my-skill
zip -r my-skill.skill .
```

生成 `my-skill.skill` 文件。

安装:

```
curl -X POST http://localhost:2026/api/skills/install \
-F "file=@my-skill.skill"
```

分享:

可以把 `.skill` 文件分享给其他人，他们安装后就能用。

21.9 Skill 开发最佳实践

1. 单一职责

一个 Skill 做一件事。不要把太多功能塞进一个 Skill。

2. 清晰的工作流

步骤要明确，顺序要合理。Agent 能按步骤执行。

3. 具体的输出格式

描述清楚输出应该是什么样。Agent 会按格式生成。

4. 参考文档

把参考资料放进 references/ 目录。Agent 需要时可以读取。

5. 测试验证

发布前测试多种场景。确保边界情况也能处理。

花叔的经验：写 Skill 就是写"方法论"。

你不需要写代码，只需要把你做某类任务的步骤和方法写清楚。Agent 会按你的步骤执行。这是最低成本的能力扩展方式。

21.10 向前桥接

自定义 Skill 清楚了。下一节，看 MCP Server 集成。

22 MCP Server 集成

22.1 先给结论

MCP (Model Context Protocol) 是 Anthropic 定义的 Agent 工具协议。

DeerFlow 支持集成 MCP Server, 扩展 Agent 能力。

支持类型:

- stdio (命令行启动)
- SSE (HTTP SSE)
- HTTP (HTTP 请求)

支持 OAuth 认证。

22.2 这节讲什么

这节讲:

1. MCP 是什么
2. 配置 MCP Server
3. 三种传输类型
4. OAuth 认证配置

22.3 MCP 是什么

MCP 是 Model Context Protocol 的缩写。

定义了 Agent 和工具之间的通信协议。

核心概念:

概念	说明
Tool	Agent 可以调用的工具
Resource	Agent 可以访问的资源
Prompt	Agent 可以使用的提示模板

MCP Server:

提供 Tools、Resources、Prompts 的服务端。

Agent 通过 MCP 协议调用。

为什么用 MCP:

- 标准化协议，一次实现多处使用
- Anthropic 推动生态，工具越来越多
- 支持多种传输方式，灵活部署

22.4 配置 MCP Server

配置位置:

`extensions_config.json`

```
{
  "mcpServers": {
    "my-mcp-server": {
      "enabled": true,
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "my-mcp-server"],
      "env": {
        "API_KEY": "$MY_API_KEY"
      },
      "description": "My MCP Server"
    }
  }
}
```

配置字段:

字段	说明	必填
enabled	是否启用	是
type	传输类型	是
description	描述	否

type 的值: `stdio`、`sse`、`http`

22.5 stdio 类型

特点:

通过命令行启动 MCP Server。通信通过 stdin/stdout。

配置示例:

```
{
  "mcpServers": {
    "web-search": {
      "enabled": true,
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@anthropic/web-search-mcp"],
      "env": {
        "TAVILY_API_KEY": "$TAVILY_API_KEY"
      }
    }
  }
}
```

字段说明:

字段	说明
command	启动命令
args	命令参数
env	环境变量

常见 MCP Server (stdio):

Server	功能
@anthropic/web-search-mcp	网页搜索
@anthropic/file-server	文件系统访问
@anthropic/sqlite-server	SQLite 数据库

22.6 SSE 类型

特点:

通过 HTTP SSE (Server-Sent Events) 通信。

配置示例:

```
{
  "mcpServers": {
    "my-sse-server": {
      "enabled": true,
      "type": "sse",
      "url": "http://my-server:8000/sse",
      "headers": {
        "Authorization": "Bearer $MY_TOKEN"
      }
    }
  }
}
```

字段说明:

字段	说明
url	SSE 端点地址
headers	请求头

适用场景:

- MCP Server 部署在远程服务器

- 需要通过 HTTP 访问

22.7 HTTP 类型

特点:

通过标准 HTTP 请求通信。

配置示例:

```
{
  "mcpServers": {
    "my-http-server": {
      "enabled": true,
      "type": "http",
      "url": "http://my-server:8000/mcp",
      "headers": {
        "Authorization": "Bearer $MY_TOKEN"
      }
    }
  }
}
```

22.8 OAuth 认证配置

适用场景:

MCP Server 需要 OAuth token 认证。

配置示例:

```
{
  "mcpServers": {
    "my-oauth-server": {
      "enabled": true,
      "type": "http",
      "url": "http://my-server:8000/mcp",
      "oauth": {
        "token_url": "https://auth.example.com/token",
        "grant_type": "client_credentials",
        "client_id": "$CLIENT_ID",
        "client_secret": "$CLIENT_SECRET"
      }
    }
  }
}
```

OAuth 字段:

字段	说明
token_url	Token 端点
grant_type	授权类型 (client_credentials / refresh_token)
client_id	客户端 ID
client_secret	客户端密钥

Token 流程:

1. DeerFlow 向 token_url 请求 token
2. 获取 access_token
3. 在请求 MCP Server 时, 自动添加 Authorization header
4. Token 过期时自动刷新

22.9 MCP 工具加载

加载机制:

- 懒加载: 首次调用时加载
- 缓存: 加载后缓存, 避免重复初始化

- 失效检测：检测配置文件变化，自动刷新

工具命名：

MCP Server 提供的工具，名称格式：

```
{server_name}_{tool_name}
```

例如：

- `web-search_search`
- `file-server_read_file`

22.10 MCP 管理 API

查看配置：

```
GET /api/mcp/config
```

更新配置：

```
PUT /api/mcp/config
Content-Type: application/json

{
  "mcpServers": {
    "new-server": {
      "enabled": true,
      "type": "stdio",
      "command": "npx",
      "args": [ "-y", "new-mcp-server" ]
    }
  }
}
```

更新后自动刷新 MCP 工具缓存。

22.11 常见 MCP Server 列表

Server	类型	功能
@anthropic/web-search-mcp	stdio	网页搜索
@anthropic/file-server	stdio	文件系统
@anthropic/sqlite-server	stdio	SQLite 数据库
@anthropic/brave-search-mcp	stdio	Brave 搜索
@anthropic/github-mcp	stdio	GitHub API

花叔的经验：MCP 是 Agent 工具生态的未来方向。

写一个 MCP Server，可以在 Claude、DeerFlow、其他 MCP 兼容 Agent 里使用。这是"一次开发，多处使用"的标准方案。如果你有内部工具要接入 Agent，强烈建议封装成 MCP Server。

22.12 向前桥接

MCP Server 集成清楚了。下一Part，讲开发指南。

Part 6: 开发指南

这Part讲怎么把 DeerFlow 当库用。不启动 HTTP 服务也能跑。

23 内嵌 Python Client

23.1 先给结论

DeerFlow 可以作为 Python 库嵌入使用。

不需要启动 HTTP 服务。

```
from deerflow.client import DeerFlowClient

client = DeerFlowClient()
response = client.chat("你好")
```

所有功能都可用：聊天、流式响应、文件上传、Memory 管理。

23.2 这节讲什么

这节讲：

1. DeerFlowClient 基本用法
2. 对话方法
3. 管理方法
4. 与 Gateway API 的区别

23.3 基本用法

导入：

```
from deerflow.client import DeerFlowClient
```

创建客户端：

```

# 默认配置
client = DeerFlowClient()

# 自定义配置路径
client = DeerFlowClient(config_path="/path/to/config.yaml")

# 自定义 extensions 配置路径
client = DeerFlowClient(
    config_path="/path/to/config.yaml",
    extensions_config_path="/path/to/extensions_config.json"
)

```

检查健康:

```

# 不适用, Client 没有健康检查
# 它是进程内调用, 不是 HTTP 服务

```

23.4 对话方法

同步对话:

```

response = client.chat(
    message="帮我分析这份报告",
    thread_id="my-thread"
)
print(response)

```

返回: 最终响应文本 (字符串)

流式对话:

```

for event in client.stream(
    message="帮我分析这份报告",
    thread_id="my-thread"
):
    if event.type == "messages-tuple":
        if event.data.get("type") == "ai":
            print(event.data["content"], end="")
    elif event.type == "end":
        print("\n完成")

```

事件类型：

事件类型	说明
values	完整状态快照
messages-tuple	单条消息更新
end	流结束

指定模型：

```
response = client.chat(  
    message="你好",  
    thread_id="my-thread",  
    model_name="gpt-4"  
)
```

指定模式：

```
# Pro 模式 (规划模式)  
response = client.chat(  
    message="帮我做一个复杂任务",  
    thread_id="my-thread",  
    is_plan_mode=True  
)  
  
# Ultra 模式 (Sub-Agent 模式)  
response = client.chat(  
    message="帮我做一份深度研究",  
    thread_id="my-thread",  
    subagent_enabled=True  
)
```

23.5 管理方法

模型管理：

```

# 列出模型
models = client.list_models()
# {"models": [{"name": "gpt-4", "display_name": "GPT-4", ...}]}

# 获取单个模型
model = client.get_model("gpt-4")
# {"name": "gpt-4", "display_name": "GPT-4", ...}

```

Skills 管理:

```

# 列出 skills
skills = client.list_skills()
# {"skills": [...]}

# 获取单个 skill
skill = client.get_skill("deep-research")

# 更新 skill 启用状态
client.update_skill("deep-research", enabled=True)

# 安装 skill
client.install_skill("/path/to/my-skill.skill")

```

MCP 管理:

```

# 获取 MCP 配置
config = client.get_mcp_config()
# {"mcp_servers": {...}}

# 更新 MCP 配置
client.update_mcp_config({
    "mcpServers": {
        "new-server": {"enabled": True, ...}
    }
})

```

Memory 管理:

```

# 获取 memory
memory = client.get_memory()

# 强制刷新
client.reload_memory()

# 获取配置
config = client.get_memory_config()

# 获取状态
status = client.get_memory_status()

```

文件上传:

```

# 上传文件
result = client.upload_files(
    thread_id="my-thread",
    files=["/path/to/report.pdf", "/path/to/data.xlsx"]
)
# {"success": true, "files": [...]}

# 列出上传文件
files = client.list_uploads("my-thread")
# {"files": [...], "count": N}

# 删除文件
client.delete_upload("my-thread", "report.pdf")

```

Artifacts:

```

# 获取 artifact
data, mime_type = client.get_artifact(
    thread_id="my-thread",
    path="outputs/report.md"
)

```

23.6 Agent 管理

重置 Agent:

```

client.reset_agent()

```

强制重新创建 Agent。用于配置变更后刷新。

状态持久化：

```
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()

response = client.chat(
    message="你好",
    thread_id="my-thread",
    checkpointer=checkpointer
)
```

23.7 与 Gateway API 的区别

维度	DeerFlowClient	Gateway API
运行方式	进程内调用	HTTP 服务
返回格式	dict / str / tuple	JSON / HTTP Response
依赖	无 HTTP 依赖	需要 HTTP 服务
并发	单进程	多进程
上传文件	本地路径	UploadFile 对象

文件上传区别：

```
# Client
client.upload_files(thread_id, ["/path/to/file.pdf"])

# Gateway API
curl -X POST /api/threads/{id}/uploads -F "files=@file.pdf"
```

Artifact 获取区别：

```

# Client: 返回 (bytes, mime_type)
data, mime_type = client.get_artifact(thread_id, path)

# Gateway API: 返回 HTTP Response
response = requests.get(f"/api/threads/{id}/artifacts/{path}")

```

23.8 使用场景

场景1: Python 脚本中使用

```

from deerflow.client import DeerFlowClient

client = DeerFlowClient()

# 做研究
response = client.chat(
    "帮我研究一下 LangGraph",
    thread_id="research-1",
    subagent_enabled=True
)

print(response)

```

场景2: 集成到其他应用**

```

# FastAPI 应用
from fastapi import FastAPI
from deerflow.client import DeerFlowClient

app = FastAPI()
client = DeerFlowClient()

@app.post("/chat")
async def chat(message: str):
    return {"response": client.chat(message)}

```

场景3: 批量处理**

```
import os

client = DeerFlowClient()

for file in os.listdir("reports/"):
    client.upload_files("batch-thread", [f"reports/{file}"])

response = client.chat(
    "分析所有上传的报告",
    thread_id="batch-thread"
)
```

花叔的经验： *DeerFlowClient* 是把 *DeerFlow* 嵌入现有系统的关键。

如果你已经有 Python 应用，不需要单独部署 *DeerFlow* 服务。直接 *import DeerFlowClient*，就能用所有功能。这是最简洁的集成方式。

23.9 向前桥接

DeerFlowClient 清楚了。下一节，看怎么自定义 Agent。

24 自定义 Agent 开发

24.1 先给结论

DeerFlow 的 Agent 系统是可扩展的。

自定义方式：

1. **自定义 Sub-Agent**：在 config.yaml 定义
2. **自定义中间件**：修改 middleware 链
3. **自定义工具**：添加新的 Python 工具
4. **自定义 Provider**：实现 SandboxProvider、GuardrailProvider

高级用法，需要深入理解 DeerFlow 架构。

24.2 这节讲什么

这节讲：

1. 自定义 Sub-Agent
2. 自定义工具
3. 自定义中间件
4. 自定义 Provider

24.3 自定义 Sub-Agent

配置方式：

```
# config.yaml
subagents:
  enabled: true
  types:
    - name: research-only
      tools: [web_search, read_file, write_file]
      description: "研究专用, 不做代码执行"

    - name: code-only
      tools: [bash, read_file, write_file]
      description: "代码执行专用"
```

使用方式:

```
task(
  description="搜索研究",
  prompt="搜索并整理信息",
  subagent_type="research-only"
)
```

内置类型:

类型	工具集
general-purpose	除 task 外所有工具
bash	只有 bash 工具

24.4 自定义工具

方式1: 在 config.yaml 定义

```
# config.yaml
tools:
  - use: my_module:my_tool
    group: custom
```

方式2: 实现 Python 工具

```

# my_module.py
from langchain_core.tools import tool

@tool
def my_custom_tool(query: str) -> str:
    """我的自定义工具。

    Args:
        query: 查询参数

    Returns:
        结果字符串
    """
    # 实现逻辑
    return f"处理结果: {query}"

```

然后在 config.yaml 引用:

```

tools:
  - use: my_module:my_custom_tool
    group: custom

```

工具分组:

```

tool_groups:
  - name: custom
    tools: [my_custom_tool]

```

24.5 自定义中间件

中间件接口:

```

from langchain_core.messages import AIMessage
from deerflow.agents.lead_agent.middleware import Middleware

class MyMiddleware(Middleware):
    async def before_model(self, state, config):
        # 在 LLM 调用前执行
        return state

    async def after_model(self, state, response, config):
        # 在 LLM 调用后执行
        return response

```

注册中间件:

修改 `packages/harness/deerflow/agents/lead_agent/agent.py` :

```

middlewares = [
    ThreadDataMiddleware(),
    ...
    MyMiddleware(), # 添加你的中间件
]

```

中间件顺序:

顺序很重要。前面的中间件先执行。

24.6 自定义 SandboxProvider

接口:

```

from deerflow.sandbox.sandbox import Sandbox, SandboxProvider

class MySandbox(Sandbox):
    def execute_command(self, command: str) -> dict:
        # 执行命令
        return {"stdout": "...", "stderr": "", "return_code": 0}

    def read_file(self, path: str) -> str:
        # 读取文件
        return "file content"

    def write_file(self, path: str, content: str):
        # 写入文件
        pass

    def list_dir(self, path: str) -> list:
        # 列出目录
        return []

class MySandboxProvider(SandboxProvider):
    async def acquire(self, thread_id: str) -> MySandbox:
        # 获取 sandbox
        return MySandbox(thread_id)

    def get(self, sandbox_id: str) -> MySandbox:
        # 获取已存在的 sandbox
        return self._sandboxes[sandbox_id]

    async def release(self, sandbox_id: str):
        # 释放 sandbox
        del self._sandboxes[sandbox_id]

```

配置:

```

sandbox:
    use: my_module:MySandboxProvider

```

24.7 自定义 GuardrailProvider

接口:

```

from deerflow.agents.lead_agent.guardrails import GuardrailProvider

class MyGuardrail(GuardrailProvider):
    async def evaluate(self, tool_call, state, config) -> dict:
        # 评估工具调用
        if tool_call.name == "dangerous_tool":
            return {
                "allowed": False,
                "reason": "禁止调用此工具"
            }
        return {"allowed": True}

```

配置:

```

guardrails:
    enabled: true
    provider: my_module:MyGuardrail

```

24.8 开发流程

第一步：理解需求

- 要扩展什么能力？
- 需要修改哪个组件？

第二步：阅读源码

位置： `packages/harness/deerflow/`

组件	目录
Agent	agents/lead_agent/
Tools	tools/
Sandbox	sandbox/
MCP	mcp/
Memory	agents/memory/

第三步：实现

按照接口定义实现。

第四步：配置

在 config.yaml 或 extensions_config.json 配置。

第五步：测试

```
cd backend
PYTHONPATH=. uv run pytest tests/test_your_feature.py -v
```

24.9 开发注意事项

1. Harness / App 边界

- Harness (`packages/harness/deerflow/`) 可发布
- App (`app/`) 不可发布
- Harness 不能 import App

CI 会检查这个边界。

2. 测试覆盖

新功能必须有测试。

```
# tests/test_my_feature.py
def test_my_feature():
    # 测试代码
    pass
```

3. 文档更新

更新 [README.md](#) 和 [CLAUDE.md](#)。

花叔的经验：自定义开发的前提是理解架构。

不建议一开始就深度定制。先用熟 DeerFlow，理解每个组件的作用，再根据实际需求定制。大部分需求通过配置和 Skills 就能满足，不需要改代码。

24.10 向前桥接

开发指南讲完了。接下来是附录。

附录

A 部署检查清单

部署前：

- ☐ Docker 正常运行
- ☐ 端口不被占用（2026、2024、8001、3000）
- ☐ 资源足够（CPU、内存、存储）
- ☐ config.yaml 配置正确
- ☐ API Key 已设置
- ☐ 安全措施已配置（生产环境）

部署后：

- ☐ 所有容器/进程正常运行
- ☐ 健康检查返回 healthy
- ☐ 能正常发消息
- ☐ Sandbox 能执行命令
- ☐ 文件能上传和读取
- ☐ 输出文件能下载

日常运维：

- ☐ 日志无异常
- ☐ 存储空间充足
- ☐ LangSmith 追踪正常（如果启用）
- ☐ 备份定时执行

B 常见问题

Q1: make dev 启动失败？

检查：

- Python/Node.js 版本是否符合要求
- uv/pnpm 是否安装

- 端口是否被占用
- config.yaml 是否正确

Q2: Docker 启动失败?

检查:

- Docker 是否运行
- 镜像是否拉取成功
- volume 是否正确挂载
- config.yaml 是否正确

Q3: Agent 响应很慢?

检查:

- 模型响应时间 (LangSmith)
- Sandbox 容器启动时间
- 是否启用 Summarization
- 网络是否正常

Q4: 文件上传失败?

检查:

- 文件大小是否超限
- 文件格式是否支持
- 存储空间是否充足

Q5: Memory 不更新?

检查:

- memory.enabled 是否为 true
- LLM 是否正常调用
- 是否有足够对话内容

Q6: Sub-Agent 超时?

检查:

- 任务是否过于复杂
- max_turns 是否足够

- 模型是否正常响应

Q7: MCP Server 不工作?

检查:

- 配置格式是否正确
- 环境变量是否设置
- MCP Server 是否正常启动

Q8: IM 渠道连不上?

检查:

- Bot Token 是否正确
- 权限是否配置正确
- 网络是否正常

C 版本更新日志

DeerFlow 2.0 (2026-02-28)

- 从头重写, 与 1.x 不共用代码
- 基于 LangGraph + LangChain
- 新增 Sub-Agents 系统
- 新增 Sandbox 隔离执行
- 新增 Memory 长期记忆
- 新增 22 个内置 Skills
- 新增 IM 渠道集成
- 新增 Claude Code 集成
- 新增内嵌 Python Client

DeerFlow 1.x (历史版本)

- Deep Research 框架
- 单场景研究工具
- 维护在 `main-1.x` 分支

D 核心概念速查表

概念	说明	位置
Lead Agent	主代理	agents/lead_agent/
Sub-Agents	子代理系统	subagents/
Sandbox	隔离执行环境	sandbox/
Skills	能力模块	skills/
Memory	长期记忆	agents/memory/
MCP	工具协议集成	mcp/
Gateway	REST API	app/gateway/
Channels	IM 渠道	app/channels/
Client	内嵌客户端	client.py

关键配置文件：

文件	用途
config.yaml	主配置
extensions_config.json	MCP 和 Skills
.env	环境变量

关键端口：

端口	服务
2026	Nginx（统一入口）
2024	LangGraph Server
8001	Gateway API
3000	Frontend

执行模式：

模式	thinking	plan_mode	subagent
Flash	false	false	false
Standard	true	false	false
Pro	true	true	false
Ultra	true	false	true

阅读指南

时间	章节	目标
Day 1	01-04	理解 DeerFlow 是什么
Day 2-3	05-11	掌握技术架构
Day 4-5	12-15	完成部署
Day 6-7	16-19	掌握核心功能
Day 8-9	20-22	Skills 进阶
Day 10	23-24	开发集成

花叔出品 | AI Native Coder · 独立开发者 公众号「花叔」/ B站「AI进化论-花生」 代表作：
Claude Code从入门到精通 · OpenClaw橙皮书 · Hermes Agent从入门到精通

DeerFlow 橙皮书

DeerFlow 2.0 从入门到精通，深度解析字节开源的 Super Agent
Harness